GPU 利用率与性能手册

GPU Utilization / Performance Handbook

原文作者：@techNmak。

先列专业术语对照表，再提供逐页完整中文译文。每个原文页面的译文之后附原文矢量对照页，完整保留公式、图表、代码和英文引用，便于核对。

正文中文：LXGW WenKai（霞鹜文楷）；英文与数字：Times New Roman。原文对照页保留原始字体。

本文采用翻译辅助服务生成初稿，统一技术术语并检查结构、字体和导航；涉及精确数值、公式或实现细节时，可直接对照相邻原文页。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
Memory Hierarchy	内存层次结构	
Shared Memory	共享内存	
Tensor Cores	张量核心	
Performance	性能	
Throughput	吞吐量	
Occupancy	占用率	
Registers	寄存器	
Latency	延迟	
Warps	线程束	
Memory Bandwidth	内存带宽	
GPU Utilization	GPU 利用率	
Synchronization	同步	
Mental Model	理解框架	
Tensor Core	张量核心	
Coalescing	合并访问	
Generation	代际	
Profiling	性能剖析	
Register	寄存器	
Resident	驻留	
Matrix	矩阵	
Tiling	分块	
Kernel	内核	
Warp	线程束	
FLOP	浮点运算	
SM	流式多处理器	SM
Streaming Multiprocessor	流式多处理器	SM
Register Pressure	寄存器压力	
Eligible Warps	可调度线程束	
Kernel Launch	内核启动	
Device Memory	设备内存	
Local Memory	局部内存	
Active Lanes	活动通道	

英文原词	中文译名	缩写／符号
Precision	精度	
Residency	驻留状态	
Spilling	溢出	
Cores	核心	
SIMT	单指令多线程	SIMT
GEMM	通用矩阵乘法	GEMM
MMA	矩阵乘加	MMA
Compute-bound	计算受限	
Memory-bound	内存受限	
Memory Traffic	内存流量	
Data Movement	数据移动	
Transformer	Transformer 模型	Transformer
FLOP Count	浮点运算次数	
Attention	注意力机制	Attention
Kernels	内核	
Tensor	张量	
Model	模型	
Matrix Multiplication	矩阵乘法	
Memory Access	内存访问	
Embedding	嵌入	
Serving	推理服务	
Thread Block	线程块	
Register File	寄存器文件	
CUDA Core	CUDA 核心	
Vector	向量	
SIMD	单指令多数据	SIMD
Pipeline	流水线	
Dispatch	分派	
Values	值	
Fused Kernels	融合内核	
Value	值	V
Working Set	工作集	
L2 Cache	二级缓存	
Memory Footprint	内存占用	

英文原词	中文译名	缩写／符号
Global Memory	全局内存	
Dense Attention	稠密注意力	
FlashAttention	FlashAttention（方法名保留）	
GPU Generations	GPU 代际	
Floating-point	浮点	
Core	核心	
Launch Overhead	启动开销	
GPU Generation	GPU 代际	
Generational	代际的	
Accumulation	累加	
Inference	推理	
Training	训练	
Scaling	缩放	
FP32	32 位浮点	FP32
Wall-clock Time	实际耗时	
Bandwidth-bound	带宽受限	
Benchmark	基准测试	
Tokens	词元	
Models	模型	
Operational Intensity	运算强度	
Arithmetic Intensity	算术强度	
Roofline Model	屋顶线模型	
Batch Size	批大小	
Normalization	归一化	
Batching	批处理	
Softmax	Softmax 归一化	Softmax
Prefill	预填充	
Query	查询	Q
Wavefront	波前	
Saturation	饱和	
CUDA Cores	CUDA 核心	
Evidence	证据	
Exact Attention	精确注意力	
Profiler	性能剖析器	

目录导航

点击章节标题跳转到中文译文，点击“原文”核对公式和图表。页码按本译本最终分页生成。

封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

GPU 利用率 / 性能（Performance） 手册 · 第 7 页 ［原文 1］

如何阅读本手册 · 第 9 页 ［原文 2］

内容 · 第 11 页 ［原文 3］

内容续 · 第 13 页 ［原文 4］

1 GPU 架构对 AI 的实际意义 · 第 15 页 ［原文 5］

2 CPU和GPU优化目标不同 · 第 17 页 ［原文 6］

3 从 内核启动（Kernel Launch） 到硬件工作 · 第 19 页 ［原文 7］

4 流式多处理器（Streaming Multiprocessor） 是执行主站 · 第 21 页 ［原文 8］

5 线程束（Warps） 和 单指令多线程（SIMT） · 第 23 页 ［原文 9］

6 背离和活动通道（Active Lanes） · 第 25 页 ［原文 10］

7 线程束（Warp） 调度程序和 可调度线程束（Eligible Warps） · 第 27 页 ［原文 11］

8 延迟（Latency）隐藏不是延迟（Latency）还原 · 第 29 页 ［原文 12］

9 驻留状态（Residency）、执行和进度 · 第 31 页 ［原文 13］

10 占用率（Occupancy）：确切定义 · 第 33 页 ［原文 14］

11 为什么 100% 占用率（Occupancy） 不是目标 · 第 35 页 ［原文 15］

12 寄存器（Registers）：快速、线程局部状态 · 第 37 页 ［原文 16］

13 寄存器压力（Register Pressure）、溢出（Spilling） 和 局部内存（Local Memory） · 第 39 页 ［原文 17］

14 共享内存（Shared Memory）：软件管理的片上存储 · 第 41 页 ［原文 18］

15 缓存：L1 和 L2 不是 共享内存（Shared Memory） · 第 43 页 ［原文 19］

16 HBM 和 设备内存（Device Memory）：容量不是带宽 · 第 45 页 ［原文 20］

17 内存层次结构（Memory Hierarchy） 是数据移动问题 · 第 47 页 ［原文 21］

18 合并访问（Coalescing）：线程束（Warp） 如何请求内存 · 第 49 页 ［原文 22］

19 执行管道：指令不使用一种资源 · 第 51 页 ［原文 23］

20 张量核心（Tensor Cores）：矩阵（Matrix） 矩阵乘加（MMA），不是魔法AI 核心（Cores） · 第 53 页 ［原文 24］

21 精度（Precision） 更改可用的执行路径 · 第 55 页 ［原文 25］

22 通用矩阵乘法（GEMM） 如何映射到 GPU 层次结构 · 第 57 页 ［原文 26］

23 指令级和线程级并行 · 第 59 页 ［原文 27］

24 同步（Synchronization） 可以暴露空闲时间 · 第 61 页 ［原文 28］

25 内核（Kernel） 启动并异步执行 · 第 63 页 ［原文 29］

26 “GPU 利用率（GPU Utilization）”的实际含义是什么 · 第 65 页 ［原文 30］

27 占用率（Occupancy），活动性和吞吐量（Throughput）不同 · 第 67 页 ［原文 31］

28 峰值 TFLOP 与实现的 吞吐量（Throughput） · 第 69 页 ［原文 32］

29 计算受限（Compute-bound） 和 内存受限（Memory-bound） 是工作负载属性 · 第 71 页 ［原文 33］
30 个 AI 工作负载对 GPU 的不同部分产生压力 · 第 73 页 ［原文 34］
31 NVIDIA 与 AMD：名称不同，原则不变 · 第 75 页 ［原文 35］
32 代架构改变了机制 · 第 77 页 ［原文 36］
33 有效执行示例：61,440 个逻辑线程 · 第 79 页 ［原文 37］
34 已工作 占用率（Occupancy） 示例：资源限制相互作用 · 第 81 页 ［原文 38］
35 种常见误解 · 第 83 页 ［原文 39］
36 实用的 理解框架（Mental Model） · 第 85 页 ［原文 40］
参考文献和进一步阅读 · 第 87 页 ［原文 41］
参考图：针对哪个问题阅读什么内容 · 第 89 页 ［原文 42］

GPU 利用率 / 性能（Performance） 手册

对应原文第 1 页；点击查看相邻原文对照页

了解 GPU 架构

从 线程束（Warps） 和 张量核心（Tensor Cores） 到 内存层次结构（Memory Hierarchy） 和 吞吐量（Throughput） 关于 AI 工作负载如何映射到真实 GPU 硬件的技术手册：流式多处理器、线程束（Warps）、调度程序、执行管道、张量核心（Tensor Cores）、寄存器（Registers）、共享内存（Shared Memory）、缓存、HBM、占用率（Occupancy）、延迟（Latency） 隐藏和 性能（Performance） 指标。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

Understanding GPU Architecture for AI

From Warps and Tensor Cores to Memory Hierarchy and Throughput

---

A technical handbook on how AI workloads map to real GPU hardware: streaming multiprocessors, warps, schedulers, execution pipelines, Tensor Cores, registers, shared memory, caches, HBM, occupancy, latency hiding, and performance metrics.

@techNmak

如何阅读本手册

对应原文第 2 页；点击查看相邻原文对照页

如何阅读本手册 当四个不同的问题分开时，GPU 架构最容易推理：

1. 存在什么工作？程序和 内核（Kernel） 定义了线程、块、指令、依赖关系和内存访问。

2. 驻留（Resident） 的作用是什么？寄存器（Registers）、共享内存（Shared Memory）、架构限制和启动配置决定了执行引擎上可以同时存在多少个块和 线程束（Warps）。

3. 准备发布什么工作？驻留（Resident） 线程束（Warp） 仍然可能因数据、依赖项、同步（Synchronization） 或其他约束而停滞。调度员在符合条件的工作中进行选择。[1,3]

4. 哪些硬件资源限制了进度？算术管道、张量核心（Tensor Cores）、内存带宽（Memory Bandwidth）、延迟（Latency）、指令问题、同步（Synchronization） 或主机端开销可能成为不同工作负载下的瓶颈。[2,3]

核心教训是 GPU 是一台 吞吐量（Throughput） 机器，而不是一个巨大的算术单元。高峰值 浮点运算（FLOP） 速率、高 占用率（Occupancy） 或显示“100% GPU 利用率（GPU Utilization）”的仪表板本身并不能证明 AI 工作负载正在高效运行。这些量衡量的是不同的事物。[2,4]

范围。本手册建立了理解人工智能系统系列其余部分所需的执行 理解框架（Mental Model）。 内存层次结构（Memory Hierarchy）、Roofline 分析、合并访问（Coalescing）、共享内存 分块（Tiling）、寄存器（Register）/占用率（Occupancy） 调整、张量核心（Tensor Core） 编程和 性能剖析（Profiling） 稍后都会收到更深入的独立手册。这里对它们的介绍仅足以使 GPU 架构保持一致。

供应商大会。 NVIDIA CUDA 术语用于具体执行路径，因为它在 AI 系统中广泛遇到。供应商特定的事实被标记。 AMD 术语用作交叉检查，以便一般 GPU 原理不会意外地呈现为 CUDA 法则。[8,9]

符号。在 NVIDIA 硬件上，线程束（Warp） 包含 32 个线程。对于具有 T 个线程的块，Nwarps =

流式多处理器（SM） 的确切资源限制、执行管道的数量和类型、缓存组织和 矩阵（Matrix） 指令因体系结构 代际（Generation） 的不同而异。[1]

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

How to read this handbook

GPU architecture is easiest to reason about when four different questions are kept separate:

1. **What work exists?** The program and kernel define threads, blocks, instructions, dependencies, and memory accesses.
2. **What work is resident?** Registers, shared memory, architectural limits, and launch configuration determine how many blocks and warps can live on an execution engine at once.
3. **What work is ready to issue?** A resident warp can still be stalled on data, dependencies, synchronization, or another constraint. Schedulers choose among eligible work.[1,3]
4. **Which hardware resource is limiting progress?** Arithmetic pipelines, Tensor Cores, memory bandwidth, latency, instruction issue, synchronization, or host-side overhead can become the bottleneck under different workloads.[2,3]

The central lesson is that a GPU is a throughput machine, not a single giant arithmetic unit. A high peak FLOP rate, high occupancy, or a dashboard showing "100% GPU utilization" does not by itself establish that an AI workload is running efficiently. Those quantities measure different things.[2,4]

> **Scope.** This handbook establishes the execution mental model needed by the rest of the Understanding AI Systems series. Memory hierarchy, Roofline analysis, coalescing, shared-memory tiling, register/occupancy tuning, Tensor Core programming, and profiling each receive deeper standalone handbooks later. Here they are introduced only far enough to make GPU architecture coherent.

Vendor convention. NVIDIA CUDA terminology is used for the concrete execution path because it is widely encountered in AI systems. Vendor-specific facts are labeled. AMD terminology is used as a cross-check so that general GPU principles are not accidentally presented as CUDA laws.[8,9]

Notation. On NVIDIA hardware, a warp contains 32 threads. For a block with T threads,

$$N_{\text{warps}} = \left\lceil \frac{T}{32} \right\rceil .$$

The exact resource limits of an SM, the number and type of execution pipelines, cache organization, and matrix instructions vary by architecture generation.[1]

内容

对应原文第 3 页；点击查看相邻原文对照页

目录 1 GPU 架构对 AI 的实际意义 5 2 CPU 和 GPU 优化不同的目标 6 3 从 内核启动（Kernel Launch） 到硬件工作 7 4 流式多处理器（Streaming Multiprocessor） 是执行中心 8 5 线程束（Warps） 和 单指令多线程（SIMT） 9 6 分歧和 活动通道（Active Lanes） 10 7 线程束（Warp） 调度程序和 可调度线程束（Eligible Warps） 11 8 延迟（Latency） 隐藏不是 延迟（Latency） 减少 12 9 驻留状态（Residency）、执行和进度 13 10 占用率（Occupancy）：确切定义 14 11 为什么 100% 占用率（Occupancy） 不是目标15 12 寄存器（Registers）：快速、线程局部状态 16 13 寄存器压力（Register Pressure）、溢出（Spilling） 和局部内存（Local Memory） 17 14 共享内存（Shared Memory）：软件管理的片上存储 18 15 高速缓存：L1 和 L2 不是 共享内存（Shared Memory） 19 16 HBM 和 设备内存（Device Memory）：容量不是带宽 20 17 内存层次结构（Memory Hierarchy） 是一个数据移动问题 21 18 合并访问（Coalescing）：线程束（Warp） 如何请求内存 22 19 执行管道：指令不使用单一资源 23 20 张量核心（Tensor Cores）： 矩阵（Matrix） 矩阵乘加（MMA），不是魔法 AI 核心（Cores） 24 21 精度（Precision） 更改可用的执行路径 25 22 通用矩阵乘法（GEMM） 如何映射到 GPU 层次结构 26 23 指令级和线程级并行性 27 24 同步（Synchronization） 可以暴露空闲时间 28

Contents

1	What GPU architecture actually means for AI	5
2	CPU and GPU optimize different objectives	6
3	From a kernel launch to hardware work	7
4	The Streaming Multiprocessor is the execution home	8
5	Warps and SIMT	9
6	Divergence and active lanes	10
7	Warp schedulers and eligible warps	11
8	Latency hiding is not latency reduction	12
9	Residency, execution, and progress	13
10	Occupancy: the exact definition	14
11	Why 100% occupancy is not the goal	15
12	Registers: fast, thread-local state	16
13	Register pressure, spilling, and local memory	17
14	Shared memory: software-managed on-chip storage	18
15	Caches: L1 and L2 are not shared memory	19
16	HBM and device memory: capacity is not bandwidth	20
17	The memory hierarchy is a data-movement problem	21
18	Coalescing: how a warp asks for memory	22
19	Execution pipelines: instructions do not use one resource	23
20	Tensor Cores: matrix MMA, not magic AI cores	24
21	Precision changes the available execution path	25
22	How GEMM maps onto the GPU hierarchy	26
23	Instruction-level and thread-level parallelism	27
24	Synchronization can expose idle time	28

内容续

对应原文第 4 页；点击查看相邻原文对照页

内容续 25 内核（Kernel） 启动和异步执行 29 26 “GPU 利用率（GPU Utilization）”实际含义是什么 30 27 占用率（Occupancy）、活动和 吞吐量（Throughput） 不同 31 28 峰值 TFLOP 与实现的 吞吐量（Throughput） 32 29 计算受限（Compute-bound） 和内存受限（Memory-bound） 是工作负载属性 33 30 AI 工作负载对 GPU 的不同部分产生压力 34 31 NVIDIA 与 AMD：名称不同，原则不变 35 32 各代架构改变了机制 36 33 有效执行示例：61,440 个逻辑线程 37 34 有效 占用率（Occupancy） 示例：资源限制相互作用 38 35 常见误解 39 36 实际操作理解框架（Mental Model） 40

参考资料和延伸阅读 41 参考图：哪个来源回答哪个问题 42 手册的前半部分解释了机器。后半部分解释了为什么常见的 性能（Performance） 数字可能不一致。这种分离是经过深思熟虑的：架构描述了存在的资源； 性能（Performance） 分析询问哪些资源实际上限制了特定工作负载。

Contents continued

25	Kernel launches and asynchronous execution	29
26	What “GPU utilization” actually means	30
27	Occupancy, activity, and throughput are different	31
28	Peak TFLOPs vs achieved throughput	32
29	Compute-bound and memory-bound are workload properties	33
30	AI workloads stress different parts of the GPU	34
31	NVIDIA vs AMD: names differ, principles persist	35
32	Architecture generations change the mechanisms	36
33	Worked execution example: 61,440 logical threads	37
34	Worked occupancy example: resource limits interact	38
35	Common misconceptions	39
36	A practical mental model	40
	References and further reading	41
	Reference map: which source answers which question	42

The first half of the handbook explains the machine. The second half explains why common performance numbers can disagree. That separation is deliberate: architecture describes what resources exist; performance analysis asks which of those resources actually limited a particular workload.

1 GPU 架构对 AI 的实际意义

对应原文第 5 页；点击查看相邻原文对照页

1 GPU 架构对 AI 的实际意义 AI 模型（Model） 表示为 张量（Tensor） 操作，但硬件不会将“Transformer 模型（Transformer） 层”或“注意力机制（Attention） 块”作为原语执行。框架、编译器和库最终将工作转化为设备操作和 内核（Kernels）。这些 内核（Kernels） 向 GPU 执行系统公开并行线程、指令、内存访问和 同步（Synchronization）。[1]

内核（Kernel） 网格块 SM 线程束（Warps） 管道 抽象是分层的，因为硬件是分层的。块提供了可以放置在 流式多处理器（SM） 上的单元。 线程束（Warps） 提供 单指令多线程（SIMT） 执行 模型（Model） 使用的本机线程分组。 寄存器（Registers） 和 共享内存（Shared Memory） 提供片上状态。缓存和 设备内存（Device Memory） 提供数据。不同的流水线执行不同的指令类。[1]

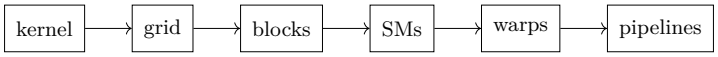
这就是为什么具有相同 浮点运算次数（FLOP Count） 的两个 内核（Kernels） 可以具有截然不同的运行时间。它们可能会生成不同的 内存流量（Memory Traffic），暴露不同数量的独立工作，使用不同的执行单元，或者遭受不同数量的 同步（Synchronization） 和分歧。 GPU 的峰值能力描述了上限。工作负载到架构的映射决定了有多少有用的工作接近这些上限。[2,3]

理解框架（Mental Model）。 GPU 架构是将并行工作加上 数据移动（Data Movement） 转变为 吞吐量（Throughput） 的机制。

AI 性能（Performance） 是工作负载-硬件映射的属性，而不是单个规格编号的属性。

1 What GPU architecture actually means for AI

An AI model is expressed as tensor operations, but hardware does not execute “a Transformer layer” or “an attention block” as a primitive. Frameworks, compilers, and libraries ultimately turn work into device operations and kernels. Those kernels expose parallel threads, instructions, memory accesses, and synchronization to a GPU execution system.[1]



The abstraction is hierarchical because the hardware is hierarchical. Blocks provide a unit that can be placed on an SM. Warps provide the native thread grouping used by the SIMT execution model. Registers and shared memory provide on-chip state. Caches and device memory feed data. Different pipelines execute different instruction classes.[1]

That is why two kernels with the same FLOP count can have very different runtimes. They may generate different memory traffic, expose different amounts of independent work, use different execution units, or suffer different amounts of synchronization and divergence. The GPU’s peak capabilities describe ceilings. The workload’s mapping onto the architecture determines how much useful work approaches those ceilings.[2,3]

Mental model. GPU architecture is the machinery that turns parallel work plus data movement into throughput. AI performance is a property of the workload-hardware mapping, not of a single specification number.

2 CPU和GPU优化目标不同

对应原文第 6 页；点击查看相邻原文对照页

2 CPU和GPU优化目标不同 CPU和GPU设计强调不同类型的进步。 CPU 投入大量资金来使相对较小的指令流集快速前进。 GPU 接受较低的每线程 性能（Performance），因此它可以保持更多的线程处于运行状态并提供高聚合 吞吐量（Throughput）。[1,2]

CPU 重点：少量线程的低 延迟（Latency） GPU 重点：来自许多并发线程的高聚合 吞吐量（Throughput） 这两个目标都不是普遍更好。串行控制密集型代码可能会导致 GPU 工作负载较差。高度并行的 矩阵（Matrix） 和 张量（Tensor） 操作可以成为出色的 GPU 工作负载，因为它们提供许多独立操作和可以映射到广泛执行资源的常规数据结构。

人工智能工作负载也包含这两种行为。密集的 矩阵乘法（Matrix Multiplication） 暴露了巨大的常规并行性。削减需要协调。 嵌入（Embedding） 查找可能涉及不规则的 内存访问（Memory Access）。

自回归 推理服务（Serving） 在每个解码步骤中可以执行少量的每个请求工作。因此，同一个 GPU 对于一个 内核（Kernel） 来说可能非常高效，而对于下一个 内核（Kernel） 来说却没有得到充分利用。

“GPU=并行，CPU=串行”太粗暴了。现代 CPU 是并行的，GPU 执行控制流。

重要的区别在于每个架构所支持的并行工作的数量、粒度和调度方式。

2 CPU and GPU optimize different objectives

CPU and GPU designs emphasize different kinds of progress. A CPU invests heavily in making a relatively small set of instruction streams advance quickly. A GPU accepts lower per-thread performance so it can keep a much larger population of threads in flight and deliver high aggregate throughput.[1,2]

CPU emphasis: low latency for a small number of threads

GPU emphasis: high aggregate throughput from many concurrent threads

Neither objective is universally better. Serial control-heavy code can be a poor GPU workload. Highly parallel matrix and tensor operations can be excellent GPU workloads because they provide many independent operations and regular data structures that can be mapped onto wide execution resources.

AI workloads also contain both kinds of behavior. Dense matrix multiplication exposes enormous regular parallelism. Reductions require coordination. Embedding lookups may involve irregular memory access. Autoregressive serving can have small per-request work at each decode step. The same GPU can therefore be spectacularly efficient for one kernel and underutilized for the next.

“GPU = parallel, CPU = serial” is too crude. Modern CPUs are parallel, and GPUs execute control flow. The important distinction is the amount, granularity, and scheduling style of parallel work each architecture is built to sustain.

3 从 内核启动（Kernel Launch） 到硬件工作

对应原文第 7 页；点击查看相邻原文对照页

3 从 内核启动（Kernel Launch） 到硬件工作 CUDA 内核启动（Kernel Launch） 创建线程块网格。每个块都包含线程，并且块被调度到流多处理器（SM）上。一旦 驻留（Resident），一个块的线程在 NVIDIA 硬件上被划分为 32 线程 线程束（Warps）。 [1]

对于包含 T 个线程的块，Nwarps =

256-线程块（Thread Block） 包含八个 线程束（Warps）。一个300-线程块（Thread Block）占用了10个线程束（Warps），最终的线程束（Warp）中只有12个有效线程。 CUDA 允许这样的块，但未使用的通道使得最终的 线程束（Warp） 效率较低。[1]

一个网格包含的块数可能远远多于同时可以容纳的 驻留（Resident） 块数。这是正常的。网格描述了逻辑工作。 GPU 在 流式多处理器（SM） 资源可用时接纳块，执行它们，并稍后接纳更多块。一个普通块在其生命周期内放置在一个 流式多处理器（SM） 上；如果资源限制允许，多个块可以是一个 流式多处理器（SM） 上的 驻留（Resident）。[1,2]

网格 块 0 块 1 块 N

程序员指定网格和块的形状。硬件/运行时决定块在何处运行，但受架构和资源限制。这种分离是 CUDA 的可扩展性理念之一：内核（Kernel） 可以描述比当前设备一次可以执行的并行工作多得多的工作。 [1]

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

3 From a kernel launch to hardware work

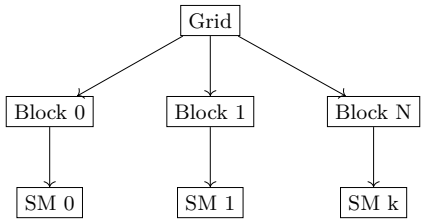
A CUDA kernel launch creates a grid of thread blocks. Each block contains threads, and blocks are scheduled onto Streaming Multiprocessors (SMs). Once resident, the threads of a block are partitioned into 32-thread warps on NVIDIA hardware.[1]

For a block containing T threads,

$$N_{\text{warps}} = \left\lceil \frac{T}{32} \right\rceil .$$

A 256-thread block contains eight warps. A 300-thread block occupies ten warps, with only 12 valid threads in the final warp. CUDA permits such a block, but the unused lanes make the final warp less efficient.[1]

A grid may contain far more blocks than can be resident simultaneously. That is normal. The grid describes the logical work. The GPU admits blocks as SM resources become available, executes them, and admits more blocks later. An ordinary block is placed on one SM for its lifetime; multiple blocks may be resident on one SM if resource limits allow.[1,2]



The programmer specifies the grid and block shape. The hardware/runtime determines where blocks run subject to architectural and resource constraints. This separation is one of CUDA's scalability ideas: a kernel can describe much more parallel work than the current device can execute at once.[1]

4 流式多处理器（Streaming Multiprocessor） 是执行主站

对应原文第 8 页；点击查看相邻原文对照页

4 流式多处理器（Streaming Multiprocessor） 是执行中心 流式多处理器（Streaming Multiprocessor） 是 NVIDIA 架构中的中央执行引擎。它不仅仅是一组算术单元。它包含保留许多 线程束（Warps） 驻留（Resident） 并取得进展所需的资源：

寄存器文件（Register File）、共享内存/L1 资源、线程束（Warp） 调度和发布机制、加载/存储路径以及多类功能单元。确切的组织因计算能力而异。[1]

有用的摘要 流式多处理器（SM） 如下所示：

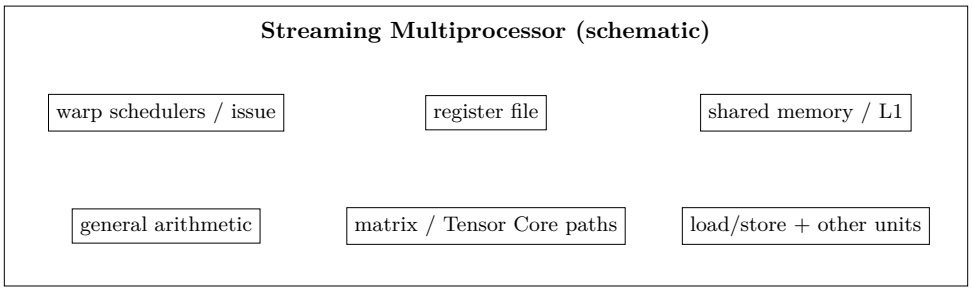
流式多处理器（Streaming Multiprocessor）（示意图） 线程束（Warp） 调度程序/问题 寄存器文件（Register File） 共享内存（Shared Memory） / L1 通用算术 矩阵（Matrix） / 张量核心（Tensor Core） 路径加载/存储 + 其他单元 内核（Kernel） 的块消耗有限的 流式多处理器（SM） 资源。 寄存器（Registers） 分配给线程。 共享内存（Shared Memory） 分配给块。架构限制 驻留（Resident） 块、线程束（Warps） 和线程。这些约束共同决定了 驻留（Resident） 可以完成多少工作。[1,2]

流式多处理器（SM） 模型（Model） 还解释了为什么“CUDA 核心（CUDA Core） 计数”是不完整的 性能（Performance） 模型（Model）。工作负载不会消耗相同 核心（Cores） 的一个通用池。不同的指令类型使用不同的管道，现代 AI 内核（Kernels） 可能严重依赖于 矩阵（Matrix） 指令、加载/存储 吞吐量（Throughput）、共享内存行为、同步（Synchronization） 和调度。有用的推理单位是整个 流式多处理器（SM） 加上它的记忆提要，而不是单个营销计数。

4 The Streaming Multiprocessor is the execution home

The Streaming Multiprocessor is the central execution engine in NVIDIA's architecture. It is not just a bundle of arithmetic units. It contains the resources needed to keep many warps resident and make progress: a register file, shared-memory/L1 resources, warp scheduling and issue machinery, load/store paths, and multiple classes of functional units. Exact organization varies by compute capability.[1]

A useful abstract SM looks like this:



A kernel's blocks consume finite SM resources. Registers are allocated for threads. Shared memory is allocated for blocks. Architectural limits cap resident blocks, warps, and threads. Those constraints jointly determine how much work can be resident.[1,2]

The SM model also explains why "CUDA core count" is an incomplete performance model. A workload does not consume one generic pool of identical cores. Different instruction types exercise different pipelines, and modern AI kernels can depend heavily on matrix instructions, load/store throughput, shared-memory behavior, synchronization, and scheduling. The useful unit of reasoning is the whole SM plus its memory feeds, not a single marketing count.

5 线程束（Warps） 和 单指令多线程（SIMT）

对应原文第 9 页；点击查看相邻原文对照页

5 线程束（Warps） 和单指令多线程（SIMT） 在NVIDIA 线程块（Thread Block） 中，线程被组织成32 个组，称为线程束（Warps）。 CUDA 使用 单指令多线程（SIMT） 描述执行：单指令、多线程。每个线程都运行相同的 内核（Kernel） 程序，但每个线程都有自己的数据，并且可以遵循不同的控制流路径。[1]

单指令多线程（SIMT） 与 单指令多数据（SIMD） 类似，因为许多通道可以一起执行公共指令，但 CUDA 明确警告这两个概念并不相同。 单指令多数据（SIMD） 公开具有一个控制流路径的固定宽度 向量（Vector） 指令。 单指令多线程（SIMT） 向程序员公开各个线程并将它们分组以供执行。[1]

编程 模型（Model） 将 线程束（Warp） 视为与 内核（Kernel） 一起进行。硬件实现细节可以更加灵活，特别是在具有独立线程调度的 NVIDIA 架构上，但可移植代码不得依赖于未记录的子 线程束（Warp） 行为。 [1]

线程束（Warp） 分组有几个后果：

- 块大小为32 的倍数可避免最终线程束（Warp） 永久填充不足；

- 相邻线程经常一起发出内存操作，这就是地址布局影响合并访问（Coalescing） 的原因；

- 控制流分歧在线程束（Warp） 内是有意义的；

- 许多线程束（Warp） 级集体操作需要显式参与掩码和同步（Synchronization） 规则。[1]

线程束（Warp） 不是“32 条独立运行的指令”。它是 32 个 CUDA 线程通常取得进展的调度/执行分组。线程仍然是逻辑程序实体； 线程束（Warp） 是面向硬件的执行组。

5 Warps and SIMT

Within an NVIDIA thread block, threads are organized into groups of 32 called warps. CUDA describes execution using SIMT: Single Instruction, Multiple Threads. Every thread runs the same kernel program, but each thread has its own data and can follow a different control-flow path.[1]

SIMT resembles SIMD because many lanes can execute a common instruction together, but CUDA explicitly warns that the two concepts are not identical. SIMD exposes a fixed-width vector instruction with one control-flow path. SIMT exposes individual threads to the programmer and groups them for execution.[1]

The programming model treats the warp as progressing together through the kernel. Hardware implementation details can be more flexible, especially on NVIDIA architectures with Independent Thread Scheduling, but portable code must not rely on undocumented sub-warp behavior.[1]

Warp grouping has several consequences:

- block sizes that are multiples of 32 avoid a permanently under-filled final warp;
- neighboring threads often issue memory operations together, which is why address layout affects coalescing;
- control-flow divergence is meaningful within a warp;
- many warp-level collective operations require explicit participation masks and synchronization rules.[1]

The warp is not “32 instructions running independently.” It is the scheduling/execution grouping through which 32 CUDA threads commonly make progress. Threads remain logical program entities; the warp is the hardware-facing execution group.

6 背离和活动通道（Active Lanes）

对应原文第 10 页；点击查看相邻原文对照页

6 分歧和 活动通道（Active Lanes） 假设 线程束（Warp） 执行数据相关分支：

路径_A();

路径_B();

如果所有活动线程选择同一分支，则 线程束（Warp） 遵循一条路径。如果某些线程选择 A 而其他线程选择 B，则 线程束（Warp） 已出现分歧。 CUDA 的执行 模型（Model） 必须考虑两条路径，同时仅启用属于当前正在执行的路径的通道。[1]

背离并不意味着 内核（Kernel） 不正确，并且它不会施加一项固定处罚。成本取决于不同路径包含的工作量、它们如何重新聚合以及特定于架构的调度。

耐用点更简单：当只有通道的子集执行指令的有用工作时，线程束（Warp） 的可用通道级 吞吐量（Throughput） 未完全使用。

分支分歧发生在 线程束（Warp） 内。两个不同的 线程束（Warps） 可以采取完全不同的路径，而不会导致它们之间出现 线程束（Warp） 分歧。[1]

Volta-代际（Generation） NVIDIA GPU 上引入的独立线程调度使线程调度比旧的锁步实现更加灵活。它不会消除任意分歧，也不会消除使用 CUDA 编程 模型（Model） 所需的 同步（Synchronization） 原语的需要。[1]

分叉也不是车道变得不活跃的唯一原因。最后的部分 线程束（Warp）、谓词指令、掩码或边界条件可以使通道处于非活动状态，而无需传统的 if/else 分割。

安全声明：分歧会减少有用的车道参与。不安全的声明：“每个不同的分支都会自动使 线程束（Warp） 慢两倍。”实际成本取决于执行的路径和硬件行为。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

6 Divergence and active lanes

Suppose a warp executes a data-dependent branch:

```
if (x > 0) {
    path_A();
} else {
    path_B();
}
```

If all active threads choose the same branch, the warp follows one path. If some threads choose A and others choose B, the warp has diverged. CUDA's execution model must account for both paths while enabling only the lanes that belong to the path currently being executed.[1]

Divergence does not mean the kernel is incorrect, and it does not impose one fixed penalty. The cost depends on how much work the different paths contain, how they reconverge, and architecturespecific scheduling. The durable point is simpler: when only a subset of lanes performs useful work for an instruction, the warp's available lane-level throughput is not fully used.

Branch divergence occurs within a warp. Two different warps can take completely different paths without causing warp divergence between them.[1]

Independent Thread Scheduling, introduced on Volta-generation NVIDIA GPUs, makes thread scheduling more flexible than older lockstep implementations. It does not make arbitrary divergence free, nor does it remove the need to use the synchronization primitives required by CUDA's programming model.[1]

Divergence is also not the only way lanes become inactive. A final partial warp, predicated instructions, masks, or boundary conditions can leave lanes inactive without a conventional if/else split.

> Safe statement: divergence can reduce useful lane participation. Unsafe statement: "every divergent branch automatically makes a warp twice as slow." The actual cost depends on the executed paths and hardware behavior.

7 线程束（Warp） 调度程序和 可调度线程束（Eligible Warps）

对应原文第 11 页；点击查看相邻原文对照页

7 线程束（Warp） 调度程序和 可调度线程束（Eligible Warps） 驻留状态（Residency） 单独并不能保证进度。 驻留（Resident） 线程束（Warp） 可能正在等待内存依赖性、算术结果、同步（Synchronization） 或其他事件。下一条指令可以发出的 线程束（Warp） 是合格的。

调度程序在 可调度线程束（Eligible Warps） 中进行选择并向适当的管道发出指令。 [3]

Nsight Compute 的调度程序 模型（Model） 非常有用，因为它区分了四种状态:

理论上的 线程束（Warps） 有效/驻留（Resident） 合格发行的 线程束（Warp） 理论上的 驻留状态（Residency） 受到启动配置和资源的限制。活动的 线程束（Warps） 是 驻留（Resident）。 可调度线程束（Eligible Warps） 是未停止的活动 线程束（Warps）。发出的 线程束（Warps） 是那些实际选择发送指令的人。

Nsight 指出，没有符合条件的 线程束（Warp） 的周期会导致跳过问题机会，并表明 延迟（Latency） 隐藏效果不佳。 [3]

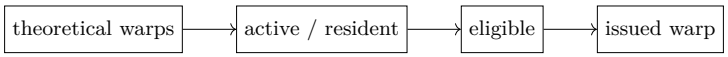
这种区别解释了一个常见的 性能剖析（Profiling） 惊喜：内核（Kernel） 可以拥有可观的 占用率（Occupancy），但仍然发行效率低下，因为在重要时刻符合资格的活动 线程束（Warps） 太少。相反，如果 内核（Kernel） 公开足够的指令级并行性并保持提供相关管道，则 内核（Kernel） 可以在中等 占用率（Occupancy） 上运行良好。

因此，时间表是建筑与表演之间的桥梁。它不能制造程序不暴露的独立性。它只能在驻留（Resident） 和 ready 的工作中进行选择。

7 Warp schedulers and eligible warps

Residency alone does not guarantee progress. A resident warp may be waiting for a memory dependency, an arithmetic result, synchronization, or another event. A warp whose next instruction can issue is eligible. Schedulers select among eligible warps and issue instructions to the appropriate pipelines.[3]

Nsight Compute's scheduler model is useful because it separates four states:



Theoretical residency is limited by launch configuration and resources. Active warps are resident. Eligible warps are active warps that are not stalled. Issued warps are those actually selected to send an instruction. Nsight notes that cycles with no eligible warp lead to skipped issue opportunities and indicate poor latency hiding.[3]

This distinction explains a common profiling surprise: a kernel can have respectable occupancy yet still issue inefficiently because too few active warps are eligible at the moments that matter. Conversely, a kernel can run well at moderate occupancy if it exposes enough instruction-level parallelism and keeps the relevant pipelines supplied.

The scheduler is therefore a bridge between architecture and performance. It cannot manufacture independence that the program does not expose. It can only choose among work that is resident and ready.

8 延迟（Latency）隐藏不是延迟（Latency）还原

对应原文第 12 页；点击查看相邻原文对照页

8 延迟（Latency）隐藏不是延迟（Latency）还原GPU操作有延迟（Latency）。全局内存请求、算术依赖性或 同步（Synchronization） 可以阻止 线程束（Warp） 立即发出下一条指令。 GPU 通过提供其他可用的独立工作来应对 延迟（Latency） 的大部分问题。[2,3]

时间 线程束（Warp） A 线程束（Warp） B 线程束（Warp） C 加载等待 使用计算 计算 线程束（Warp） A 的内存 延迟（Latency） 尚未消失。机器只是在部分等待间隔期间完成了 B 和 C 的有用工作。这是隐藏的延迟（Latency）。[2]

两种独立帮助：

- 线程级并行（TLP）：其他线程束（Warps）有就绪指令；

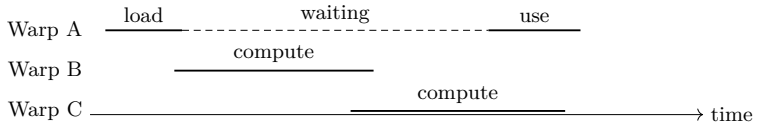
- 指令级并行性(ILP)：同一个线程束（Warp） 具有独立的操作，无需等待停止的结果即可继续进行。

这就是为什么 占用率（Occupancy） 很重要，但并不是故事的全部。更多 驻留（Resident） 线程束（Warps） 为 TLP 创建更多候选。但是，如果 内核（Kernel） 已经有足够的准备工作来覆盖重要的延迟，那么添加 驻留状态（Residency） 可能不会改善运行时间。 [2]

GPU 通常会获胜，不是因为每个 内存访问（Memory Access） 的 延迟（Latency） 都很低，而是因为架构的构建是为了通过维护和调度许多独立操作来容忍 延迟（Latency）。

8 Latency hiding is not latency reduction

GPU operations have latency. A global-memory request, an arithmetic dependency, or synchronization can prevent one warp from issuing its next instruction immediately. GPUs cope with much of this latency by having other independent work available.[2,3]



Warp A's memory latency has not vanished. The machine simply did useful work from B and C during part of the waiting interval. This is latency hiding.[2]

Two kinds of independence help:

- thread-level parallelism (TLP): other warps have ready instructions;
- instruction-level parallelism (ILP): the same warp has independent operations that can proceed without waiting on the stalled result.

This is why occupancy matters but is not the whole story. More resident warps create more candidates for TLP. But if a kernel already has enough ready work to cover the important latencies, adding residency may not improve runtime.[2]

> A GPU often wins not because each memory access is low latency, but because the architecture is built to tolerate latency by maintaining and scheduling many independent operations.

9 驻留状态（Residency）、执行和进度

对应原文第 13 页；点击查看相邻原文对照页

9 驻留状态（Residency）、执行和进度 这三个经常被混淆在一起的词应该分开。

驻留（Resident） 表示块或 线程束（Warp） 已被接纳到 流式多处理器（SM） 并且其所需的状态已分配。 寄存器（Registers）、共享内存（Shared Memory）、块槽、线程束（Warp） 槽和线程限制约束 驻留状态（Residency）。[1,2]

Active 通常由 性能剖析（Profiling） 工具用于尚未完成的 驻留（Resident） 线程束（Warps）。活动的 线程束（Warp） 可能仍会花费停滞的周期。[3]

合格意味着 驻留（Resident） 线程束（Warp） 已准备好发出其下一条指令，而不是被依赖项或 同步（Synchronization） 条件阻止。已发出意味着调度程序实际上在该调度机会中选择了 线程束（Warp） 到 分派（Dispatch） 指令。[3]

逻辑工作 ⊃ 驻留（Resident） 工作 ⊃ 合格工作 ⊃ 及时发布的工作。

内核（Kernel） 可以启动数百万个逻辑线程，而 驻留（Resident） 只能是有界集合。它可以有许多 驻留（Resident） 线程束（Warps），但只有少数符合条件。当特定执行 流水线（Pipeline） 饱和而另一个执行空闲时，它可以具有 可调度线程束（Eligible Warps）。

在解释 性能（Performance） 计数器时，这种分离至关重要。 驻留状态（Residency） 指标无法告诉您哪些功能单元正忙。基于“任何 内核（Kernel） 执行”的利用率指标无法告诉您主要的 流水线（Pipeline） 是否接近峰值。一个子系统的 吞吐量（Throughput） 指标无法告诉您端到端应用程序时间是否由另一子系统主导。[3,4]

9 Residency, execution, and progress

Three words that are often blurred together should stay separate.

Resident means a block or warp has been admitted to an SM and its required state is allocated. Registers, shared memory, block slots, warp slots, and thread limits constrain residency.[1,2]

Active is commonly used by profiling tools for resident warps that have not completed. An active warp may still spend cycles stalled.[3]

Eligible means a resident warp is ready to issue its next instruction rather than being blocked by a dependency or synchronization condition. Issued means the scheduler actually selected that warp to dispatch an instruction in that scheduling opportunity.[3]

$$\text{logical work} \supset \text{resident work} \supset \text{eligible work} \supset \text{issued work at a moment in time.}$$

A kernel can launch millions of logical threads while only a bounded set is resident. It can have many resident warps while only a few are eligible. It can have eligible warps while a particular execution pipeline is saturated and another is idle.

This separation is essential when interpreting performance counters. A residency metric cannot tell you which functional units were busy. A utilization metric based on “any kernel executing” cannot tell you whether the dominant pipeline was near peak. A throughput metric for one subsystem cannot tell you whether end-to-end application time was dominated by another subsystem.[3,4]

10 占用率（Occupancy）：确切定义

对应原文第 14 页；点击查看相邻原文对照页

10 占用率（Occupancy）：占用率（Occupancy） 的确切定义是 驻留状态（Residency） 指标。在 CUDA 中，它是根据 流式多处理器（SM） 上活动的 线程束（Warps） 数量相对于该 流式多处理器（SM） 活动 线程束（Warps） 的架构限制来计算的。[2]

占用率（Occupancy） = 活动 驻留（Resident） 线程束（Warps） 每个 流式多处理器（SM） 最大 驻留（Resident） 线程束（Warps） 每个 流式多处理器（SM）。

如果架构允许 流式多处理器（SM） 上最多有 64 个 驻留（Resident） 线程束（Warps），并且 内核（Kernel） 配置允许 32 个 驻留（Resident），则理论上的 占用率（Occupancy） 为 50%。

占用率（Occupancy） 受到多种有限资源的共同约束：

- 每个块的线程数和最大驻留（Resident） 线程数；

- 每块线程束（Warps） 和最大驻留（Resident） 线程束（Warps）；

- 最大驻留（Resident） 块；

- 寄存器（Registers） 由驻留（Resident） 线程消耗；

- 共享内存（Shared Memory） 被驻留（Resident） 块消耗；

- 特定于架构的资源分配粒度。[1,2]

一种有限的资源足以阻止其他块变成 驻留（Resident）。几乎不使用 共享内存（Shared Memory） 的 内核（Kernel） 仍然可能受到 寄存器（Register） 限制。适度使用 寄存器（Register） 的 内核（Kernel） 可以受到共享内存限制。

非常小的块在填充可用线程槽之前可能会达到块计数限制。[1]

因此，占用率（Occupancy） 是 驻留状态（Residency） 比率，而不是算术单元执行有用工作的百分比。Nsight Compute 甚至提供了专用的 占用率（Occupancy） 计算器，因为资源限制和分配粒度的相互作用可能不明显。 [3]

占用率（Occupancy） 回答“这次发射能维持多少 线程束（Warp） 驻留状态（Residency）？”它没有回答“这个 内核（Kernel） 与峰值计算 吞吐量（Throughput） 有多接近？”

10 Occupancy: the exact definition

Occupancy is a residency metric. In CUDA it is computed from the number of warps active on an SM relative to that SM's architectural limit for active warps.[2]

$$\text{occupancy} = \frac{\text{active resident warps per SM}}{\text{maximum resident warps per SM}}.$$

If an architecture permits at most 64 resident warps on an SM and a kernel configuration allows 32 to be resident, the theoretical occupancy is 50%.

Occupancy is constrained jointly by several finite resources:

- threads per block and maximum resident threads;
- warps per block and maximum resident warps;
- maximum resident blocks;
- registers consumed by resident threads;
- shared memory consumed by resident blocks;
- architecture-specific resource allocation granularity.[1,2]

One limiting resource is enough to stop additional blocks from becoming resident. A kernel that barely uses shared memory can still be register-limited. A kernel with modest register use can be shared-memory-limited. Very small blocks can hit the block-count limit before filling the available thread slots.[1]

Occupancy is therefore a residency ratio, not the percentage of arithmetic units doing useful work. Nsight Compute even provides a dedicated Occupancy Calculator because the interaction of resource limits and allocation granularity can be non-obvious.[3]

> Occupancy answers "how much warp residency can this launch sustain?" It does not answer "how close is this kernel to peak compute throughput?"

11 为什么 100% 占用率（Occupancy） 不是目标

对应原文第 15 页；点击查看相邻原文对照页

11 为什么100% 占用率（Occupancy） 不是目标 更多驻留（Resident） 线程束（Warps） 可以提高延迟（Latency） 容差，但性能（Performance） 不会随占用率（Occupancy） 无限期上升。

一旦 内核（Kernel） 已经暴露了足够的准备工作来覆盖其重要的摊位，额外的 驻留状态（Residency） 可能会增加一点点。 [2]

尝试强制 占用率（Occupancy） 向上也可以产生权衡。 寄存器（Registers） 很有用，因为它们在芯片上保留线程本地状态。减少允许的 寄存器（Register） 计数可能会接纳更多块，但它也会增加指令计数或将 值（Values） 溢出到 局部内存（Local Memory）。具有足够 ILP 的较低 占用率（Occupancy） 内核（Kernel） 可能会优于会溢出或执行更多工作的较高 占用率（Occupancy） 版本。 [2]

较高的 占用率（Occupancy）⇏ 按比例较高的 性能（Performance）。

低 占用率（Occupancy） 仍可诊断。如果 线程束（Warps） 和 驻留（Resident） 的数量太少，则当一个线程束（Warp） 停顿时，调度程序的替代方案就会较少。但正确的问题不是“如何最大化 占用率（Occupancy）？”是“现成工作不够充分，阻碍了瓶颈资源的有效利用吗？”性能剖析（Profiling） 应将 占用率（Occupancy） 连接到调度程序行为和资源 吞吐量（Throughput）。如果由于没有 可调度线程束（Eligible Warps） 而经常跳过问题槽，则更多 延迟（Latency） 隐藏容量可能会有所帮助。如果相关的计算或内存路径已经接近其可持续极限，则提高 占用率（Occupancy） 可能不会起任何作用。 [3]

将 占用率（Occupancy） 视为约束和诊断，而不是分数。最快的内核（Kernel）为目标；最大 占用率（Occupancy） 只是达到此目的的一种可能方法。

11 Why 100% occupancy is not the goal

More resident warps can improve latency tolerance, but performance does not rise indefinitely with occupancy. Once a kernel already exposes enough ready work to cover its important stalls, additional residency may add little.[2]

Trying to force occupancy upward can also create a trade-off. Registers are useful because they keep thread-local state on chip. Reducing the allowed register count may admit more blocks, but it can also increase instruction count or spill values to local memory. A lower-occupancy kernel with enough ILP may outperform a higher-occupancy version that spills or does more work.[2]

$$\text{higher occupancy} \not\Rightarrow \text{proportionally higher performance.}$$

Low occupancy can still be diagnostic. If too few warps are resident, the scheduler has fewer alternatives when one warp stalls. But the right question is not “how do I maximize occupancy?” It is “is insufficient ready work preventing the bottlenecked resource from being used effectively?”

Profiling should connect occupancy to scheduler behavior and resource throughput. If issue slots are frequently skipped because there are no eligible warps, more latency-hiding capacity may help. If the relevant compute or memory path is already near its sustainable limit, raising occupancy may do nothing.[3]

> Treat occupancy as a constraint and diagnostic, not a score. The fastest kernel is the goal; maximum occupancy is only one possible means to that end.

12 寄存器（Registers）：快速、线程局部状态

对应原文第 16 页；点击查看相邻原文对照页

12 寄存器（Registers）：快速、线程本地状态 寄存器（Registers） 是与 流式多处理器（SM） 关联的片上存储，并且逻辑上为每个线程私有。编译器将它们用于线程本地 值（Values），例如循环变量、地址、累加器和中间算术状态。[1,2]

寄存器（Registers） 之所以重要有两个相反的原因。首先，它们正是 性能（Performance） 敏感的 值（Values） 通常应该存在的地方：接近执行，而不需要重复传输到更大的内存。其次，寄存器文件（Register File） 是有限的，并在 流式多处理器（SM） 上的所有 驻留（Resident） 线程之间共享。因此，每线程 寄存器（Register） 的高使用量可以减少同时适合的块和 线程束（Warps） 的数量。[1,2]

这就是为什么 寄存器（Register） 的使用既是一个容量问题，又是一个 延迟（Latency） 问题。 内核（Kernel） 可以通过在 寄存器（Registers） 中保留更多 值（Values） 来提高本地重用，但会丢失 驻留状态（Residency），因为适合的线程上下文较少。最佳点是特定于工作负载的。

寄存器（Registers） 也参与依赖链。如果一条指令消耗了前一条指令生成的 值（Value），则使用者可能需要等待，直到结果可用。其他独立指令或线程束（Warps）可以隐藏延迟（Latency）。 NVIDIA 的最佳实践指南专门讨论了通过足够的并行工作隐藏 寄存器（Register） 依赖项。 [2]

对于 AI 内核（Kernels）、寄存器（Register）、分块（Tiling） 是高 性能（Performance）、矩阵乘法（Matrix Multiplication） 和 融合内核（Fused Kernels） 的核心，因为部分结果可以在算术管道附近累积。但激进的使用 寄存器（Register） 会增加压力。后续手册将定量地处理 寄存器（Register）/占用率（Occupancy） 的权衡；这里的架构教训是，寄存器（Registers） 既是高 值（Value） 资源，又是有限的 驻留状态（Residency） 预算。

12 Registers: fast, thread-local state

Registers are on-chip storage associated with an SM and logically private to each thread. The compiler uses them for thread-local values such as loop variables, addresses, accumulators, and intermediate arithmetic state.[1,2]

Registers matter for two opposite reasons. First, they are exactly where performance-sensitive values often should live: close to execution, without repeated traffic to larger memory. Second, the register file is finite and shared among all resident threads on an SM. High per-thread register use can therefore reduce how many blocks and warps fit concurrently.[1,2]

This is why register usage is a capacity problem as well as a latency problem. A kernel can improve local reuse by holding more values in registers but lose residency because fewer thread contexts fit. The optimal point is workload-specific.

Registers also participate in dependency chains. If an instruction consumes a value produced by a previous instruction, the consumer may need to wait until the result is available. Other independent instructions or warps can hide that latency. NVIDIA's Best Practices Guide specifically discusses hiding register dependencies through sufficient parallel work.[2]

For AI kernels, register tiling is central to high-performance matrix multiplication and fused kernels because partial results can be accumulated close to the arithmetic pipelines. But aggressive register use increases pressure. Later handbooks will treat the register/occupancy trade-off quantitatively; here the architectural lesson is that registers are both a high-value resource and a finite residency budget.

13 寄存器压力（Register Pressure）、溢出（Spilling）和 局部内存（Local Memory）

对应原文第 17 页；点击查看相邻原文对照页

13 寄存器压力（Register Pressure）、溢出（Spilling） 和 局部内存（Local Memory） 当 寄存器（Register） 需求太高时，或者当某些线程局部对象无法保留在 寄存器（Registers） 中时，编译器可以将它们放置在 CUDA 局部内存（Local Memory） 中。尽管有这个名称，但此存储仅在可见性方面是按线程的；它由 设备内存（Device Memory） 支持，而不是由私有片上存储器支持。[2]

寄存器（Register） 溢出（Spilling） 是 值（Values） 达到 局部内存（Local Memory） 的常见原因之一。CUDA 还可能在那里放置一些大型或动态索引的线程本地对象。这些访问通过设备内存和缓存路径，因此它们比 寄存器（Register） 访问昂贵得多，并且可以增加大量的 内存流量（Memory Traffic）。[2]

减少 寄存器（Register） 计数 可能的后果 更多的块可能会变成 驻留（Resident） 占用率（Occupancy） 可能会增加更少的片上线程本地容量 值（Values） 可能会溢出更多的本地 -内存流量（Memory Traffic） 延迟（Latency） / 带宽成本可能会上升 这就是为什么 maxrregcount 或启动界限是调整控制，而不是自动改进。 NVIDIA 明确警告，较低的 寄存器（Register） 限制可能会增加 溢出（Spilling），即使在增加 占用率（Occupancy） 的同时也是如此。[2]

实际工作流程是检查编译器资源报告并分析生成的 内核（Kernel），而不是假设更少的 寄存器（Registers） 或更多的 占用率（Occupancy） 一定更好。

13 Register pressure, spilling, and local memory

When register demand is too high, or when certain thread-local objects cannot be kept in registers, the compiler can place them in CUDA local memory. Despite the name, this storage is per-thread only in visibility; it is backed by device memory rather than by a private on-chip memory.[2]

Register spilling is one common reason values reach local memory. CUDA may also place some large or dynamically indexed thread-local objects there. These accesses travel through the device-memory and cache path, so they are far more expensive than register access and can add substantial memory traffic.[2]

Reduce register count	Possible consequence
more blocks may become resident	occupancy may increase
less on-chip thread-local capacity	values may spill
more local-memory traffic	latency / bandwidth cost may rise

That is why `-maxrregcount` or launch bounds are tuning controls, not automatic improvements. NVIDIA explicitly warns that lower register limits can increase spilling even while increasing occupancy.[2]

The practical workflow is to inspect compiler resource reports and profile the resulting kernel, rather than assuming that fewer registers or more occupancy must be better.

14 共享内存（Shared Memory）：软件管理的片上存储

对应原文第 18 页；点击查看相邻原文对照页

14 共享内存（Shared Memory）：软件管理的片上存储 共享内存（Shared Memory） 是显式寻址的片上存储，块内的线程可用于协作和数据重用。与硬件管理的缓存不同，软件选择在那里放置什么以及何时在其周围同步。[1,2]

常见模式是 分块（Tiling）：将较大 张量（Tensor） 的子集从 设备内存（Device Memory） 加载到 共享内存（Shared Memory），如有必要，进行同步，重复使用该图块进行多个操作，然后继续。目标不仅仅是使用“更快的内存”。目标是减少从层次结构中较远的位置进行昂贵的重复移动。

共享内存（Shared Memory） 还消耗 流式多处理器（SM） 驻留状态（Residency） 预算。请求大量分配的块可能会阻止其他块同时成为 驻留（Resident）。因此，共享内存（Shared Memory） 可以提高局部性，同时减少 占用率（Occupancy）。这两种效果都是真实存在的。[1,2]

在最新的 NVIDIA 架构上，共享内存（Shared Memory） 和 L1 功能可以共享或分区统一的片上数据缓存资源，但确切的大小和组织因计算能力而异。该实现事实不会将 共享内存（Shared Memory） 转变为普通缓存：程序员仍然显式分配和寻址 共享内存（Shared Memory）。[1]

同步（Synchronization） 是成本 模型（Model） 的另一部分。如果线程通过 共享内存（Shared Memory） 进行协作，则可能需要诸如 __syncthreads() 之类的屏障来确保正确性。因此，必须权衡数据重用效益与存储压力、银行访问行为和 同步（Synchronization） 开销。

14 Shared memory: software-managed on-chip storage

Shared memory is explicitly addressed on-chip storage that threads within a block can use for cooperation and data reuse. Software chooses what to place there and when to synchronize around it, unlike a hardware-managed cache.[1,2]

A common pattern is tiling: load a subset of a larger tensor from device memory into shared memory, synchronize if necessary, reuse that tile for multiple operations, then move on. The goal is not simply to use a “faster memory.” The goal is to reduce expensive repeated movement from farther away in the hierarchy.

Shared memory also consumes an SM residency budget. A block that requests a large allocation may prevent other blocks from being resident simultaneously. Thus shared memory can improve locality while reducing occupancy. Both effects are real.[1,2]

On recent NVIDIA architectures, shared memory and L1 functionality can share or partition a unified on-chip data-cache resource, but the exact size and organization vary by compute capability. That implementation fact does not turn shared memory into an ordinary cache: the programmer still allocates and addresses shared memory explicitly.[1]

Synchronization is another part of the cost model. If threads cooperate through shared memory, barriers such as `__syncthreads()` may be required for correctness. The data reuse benefit must therefore be weighed against storage pressure, bank-access behavior, and synchronization overhead.

15 缓存：L1 和 L2 不是 共享内存（Shared Memory）

对应原文第 19 页；点击查看相邻原文对照页

15 缓存：L1 和 L2 不是 共享内存（Shared Memory） GPU 还包含硬件管理的缓存。在 CUDA 的 设备内存（Device Memory） 模型（Model） 中，全局 内存流量（Memory Traffic） 可以通过缓存结构，包括每流式多处理器（SM） L1 功能和设备范围的 二级缓存（L2 Cache）。确切的政策和能力取决于架构。[1]

关键区别在于控制：

共享内存（Shared Memory） 程序中显式的缓存管理 硬件管理范围 透明层次结构 使用有意的暂存/重用时间/空间局部性 驻留状态（Residency） 分配可以限制块 无每块共享内存预算 缓存可以减少到达 HBM/设备 DRAM 的流量，但不能保证保留每个有用的 工作集（Working Set）。访问模式、重用距离、争用和架构都很重要。

这就是为什么“张量（Tensor） 适合 GPU 内存”几乎没有提及 内核（Kernel） 性能（Performance）。容量告诉我们数据是否可以驻留在内存系统中的某个位置。 性能（Performance） 取决于数据移动的频率、芯片上捕获的重用程度以及 工作集（Working Set） 和访问模式是否与硬件匹配。

以后的手册将对层次结构进行定量处理。对于 AS01，请记住这些角色：寄存器（Registers） 是线程局部状态，共享内存（Shared Memory） 是显式片上协作/重用，缓存是硬件管理的局部性，设备内存（Device Memory） 在距执行管道更远的距离处提供大容量。

块本地；集群 DSM（如果支持）

15 Caches: L1 and L2 are not shared memory

GPUs also contain hardware-managed caches. In CUDA's device memory model, global memory traffic can pass through cache structures, including per-SM L1 functionality and a device-wide L2 cache. Exact policies and capacities depend on architecture.[1]

The key distinction is control:

	Shared memory	**Cache**
management	explicit in program	hardware-managed
scope	block-local; cluster DSM if supported	transparent hierarchy
use	deliberate staging / reuse	temporal / spatial locality
residency	allocation can limit blocks	no per-block shared-memory budget

Caches can reduce the traffic that reaches HBM/device DRAM, but they are not guaranteed to hold every useful working set. Access pattern, reuse distance, contention, and architecture all matter.

This is why "the tensor fits in GPU memory" says almost nothing about kernel performance. Capacity tells us whether data can reside somewhere in the memory system. Performance depends on how often data moves, how much reuse is captured on chip, and whether the working set and access pattern match the hardware.

Later handbooks will treat the hierarchy quantitatively. For AS01, remember the roles: registers are thread-local state, shared memory is explicit on-chip cooperation/reuse, caches are hardwaremanaged locality, and device memory provides large capacity at much greater distance from the execution pipelines.

16 HBM 和 设备内存（Device Memory）：容量不是带宽

对应原文第 20 页；点击查看相邻原文对照页

16 HBM 和 设备内存（Device Memory）：容量不是带宽 CUDA 的编程 模型（Model） 使用 全局内存（Global Memory） 作为所有 SM 可见的设备附加 DRAM。许多数据中心人工智能加速器通过 HBM 技术实现这种大容量内存，因此人工智能系统讨论经常使用“HBM”作为远大容量 GPU 内存的简写。[5,10]

两个数量必须分开：

- 容量：可以存储多少字节；

- 带宽：在给定的访问模式和工作负载下每秒可以传输多少字节。

模型（Model） 可以完全放入 设备内存（Device Memory） 中，但仍然运行缓慢，因为 内核（Kernels） 通过 HBM 接口重复移动大量数据。相反，内核（Kernel） 可能具有较小的 内存占用（Memory Footprint） 但对 延迟（Latency） 敏感，因为它的访问是稀疏的或相关的。

峰值内存带宽（Memory Bandwidth）也是一个上限，而不是一个承诺。有效带宽取决于访问 合并访问（Coalescing）、缓存行为、指令混合、未完成的请求和其他详细信息。 CUDA 的最佳实践指南建议测量有效带宽，而不是假设已达到理论规范。 [2]

“适合 HBM”是一种能力声明。“HBM 已饱和”是一个带宽声明。“内核（Kernel） 正在等待内存”是 延迟（Latency）/依赖项语句。它们是相关的但不能互换。

16 HBM and device memory: capacity is not bandwidth

CUDA's programming model uses global memory for the device-attached DRAM visible to all SMs. Many data-center AI accelerators implement this large memory with HBM technology, so AI systems discussions often use "HBM" as shorthand for the far, high-capacity level of GPU memory.[5,10]

Two quantities must be separated:

- capacity: how many bytes can be stored;
- bandwidth: how many bytes per second can be transferred under a given access pattern and workload.

A model can fit entirely in device memory and still run slowly because kernels repeatedly move large amounts of data through the HBM interface. Conversely, a kernel may have a small memory footprint yet be latency-sensitive because its accesses are sparse or dependent.

Peak memory bandwidth is also a ceiling, not a promise. Effective bandwidth depends on access coalescing, cache behavior, instruction mix, outstanding requests, and other details. CUDA's Best Practices Guide recommends measuring effective bandwidth rather than assuming the theoretical specification is achieved.[2]

> "Fits in HBM" is a capacity statement. "HBM is saturated" is a bandwidth statement. "The kernel is waiting on memory" is a latency/dependency statement. They are related but not interchangeable.

17 内存层次结构（Memory Hierarchy） 是数据移动问题

对应原文第 21 页；点击查看相邻原文对照页

17 内存层次结构（Memory Hierarchy） 是一个数据移动问题 一个有用的面向 AI 的 内存层次结构（Memory Hierarchy） 是：

寄存器（Registers） - 线程本地，在 流式多处理器（SM） 上 共享内存（Shared Memory） / L1 资源 - 在 流式多处理器（SM） 上 二级缓存（L2 Cache） - 设备范围 HBM / 全局 设备内存（Device Memory） 层次结构不仅仅是“快”和“慢”位置的排名。每个级别都有不同的容量、范围、管理策略、带宽和访问语义。 性能（Performance） 问题是在级别之间移动了多少字节以及从附近的数据中提取了多少有用的计算。

同样的想法也出现在 通用矩阵乘法（GEMM） 分块（Tiling）、元素融合 内核（Kernels）、卷积、归约和许多编译器优化中。重复使用很重要，因为移动 值（Value） 一次并多次使用它比从更远的地方重复获取相同的 值（Value） 更便宜。

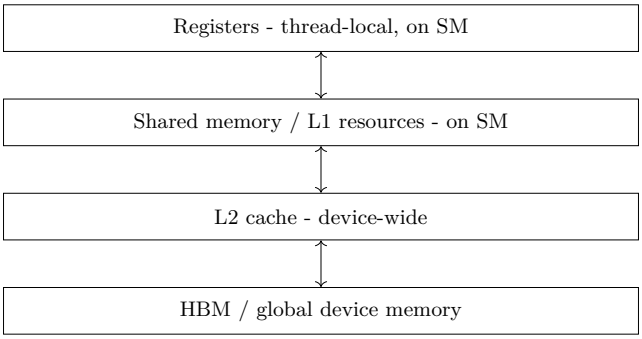
这也解释了为什么 浮点运算次数（FLOP Count） 本身是一个很差的运行时预测器。两种实现可以执行类似的算术，但生成不同的 内存流量（Memory Traffic）。工作负载的 数据移动（Data Movement） 是其执行算法的一部分，即使纸上的数学表达式根本没有提到内存。

FlashAttention 是一个有用的示例：它计算精确的 稠密注意力（Dense Attention） 但更改执行计划。

中间块在计算单元附近被消耗和重用，而不是通过 HBM 重复往返大型中间体。 [12]

17 The memory hierarchy is a data-movement problem

A useful AI-oriented memory hierarchy is:



The hierarchy is not merely a ranking of “fast” and “slow” locations. Each level has different capacity, scope, management policy, bandwidth, and access semantics. The performance question is how many bytes move between levels and how much useful computation is extracted from data while it is nearby.

FlashAttention is a useful example: it computes exact dense attention but changes the execution schedule. Intermediate tiles are consumed and reused near the compute units instead of repeatedly round-tripping large intermediates through HBM.[12]

The same idea appears in GEMM tiling, fused elementwise kernels, convolutions, reductions, and many compiler optimizations. Reuse matters because moving a value once and using it many times can be cheaper than fetching the same value repeatedly from farther away.

This also explains why FLOP count alone is a poor runtime predictor. Two implementations can perform similar arithmetic but generate different memory traffic. A workload’s data movement is part of its execution algorithm, even when the mathematical expression on paper does not mention memory at all.

18 合并访问（Coalescing）：线程束（Warp） 如何请求内存

对应原文第 22 页；点击查看相邻原文对照页

18 合并访问（Coalescing）：线程束（Warp） 如何请求内存 当 线程束（Warp） 中的线程发出全局内存访问时，GPU 将其请求合并到内存事务中。 CUDA 将此称为 合并访问（Coalescing）。高效访问使用每个事务引入的字节，而不是获取包含少量请求数据的许多段。[1,2]

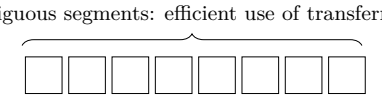
几个连续的段：有效利用传输的字节 分散的模式可能需要 线程束（Warp） 从未使用的更多事务和传输字节。即使程序请求相同数量的有用 值（Values），这也会浪费带宽。[2]

因此，合并访问（Coalescing） 将线程映射连接到内存 性能（Performance）。读取相同的 张量（Tensor） 可能很便宜也可能很昂贵，具体取决于相邻通道遍历的维度以及数据的布局方式。

AS07将对此进行深入处理； AS01 只需要了解架构：内存流量（Memory Traffic） 是由线程组共同生成的，因此地址模式很重要。

18 Coalescing: how a warp asks for memory

When threads in a warp issue global-memory accesses, the GPU combines their requests into memory transactions. CUDA calls this coalescing. Efficient access uses the bytes brought in by each transaction rather than fetching many segments that contain little requested data.[1,2]



A scattered pattern can require more transactions and transfer bytes that the warp never uses. This wastes bandwidth even though the program requested the same number of useful values.[2]

Coalescing therefore connects thread mapping to memory performance. The same tensor can be cheap or expensive to read depending on which dimension neighboring lanes traverse and how the data is laid out. AS07 will treat this deeply; AS01 only needs the architectural insight: memory traffic is generated collectively by groups of threads, so address pattern matters.

19 执行管道：指令不使用一种资源

对应原文第 23 页；点击查看相邻原文对照页

19 执行管道：指令不使用一种资源 流式多处理器（SM） 包含多个执行路径。 浮点（Floating-point） 加法、整数运算、加载/存储指令、特殊函数和 矩阵（Matrix） 乘法累加并非全部由一个无差别的“核心（Core）”提供服务。精确的 流水线（Pipeline） 计数和映射在 GPU 代际（GPU Generations） 中发生变化。[1,5,6]

这很重要，因为利用率可能是不对称的。 内核（Kernel） 可能会使一个子系统饱和，而其他子系统大部分处于空闲状态：

- 密集的通用矩阵乘法（GEMM） 可能会花费大量的有用时间来喂养矩阵（Matrix） 乘法硬件；

- 内存复制可以接近内存系统限制，同时执行很少的算术；

- 元素内核（Kernel） 可以受内存流量（Memory Traffic） 或指令问题而非张量核心（Tensor Cores） 的限制；

- 减少可能受到相关性和同步（Synchronization） 的限制。

因此，“GPU 有未使用的计算”并不是一个完整的诊断。哪个计算？哪个指令路径？内存已经是瓶颈了吗？有可调度线程束（Eligible Warps）吗？ 内核（Kernel） 是否太小而无法暴露足够的设备范围并行性？

Nsight Compute 的 吞吐量（Throughput） 模型（Model） 通过报告相对于峰值的高级计算和内存 吞吐量（Throughput） 并暴露较低级别的贡献者来反映这一现实。 [3]

GPU 是耦合子系统的集合。 性能（Performance） 调整意味着找到限制特定工作负载有用进度的子系统，而不是试图使图中的每个块都读取 100%。

19 Execution pipelines: instructions do not use one resource

An SM contains multiple execution paths. A floating-point add, integer operation, load/store instruction, special function, and matrix multiply-accumulate are not all serviced by one undifferentiated “core.” Exact pipeline counts and mappings change across GPU generations.[1,5,6]

This matters because utilization can be asymmetric. A kernel might saturate one subsystem while others are mostly idle:

- a dense GEMM may spend much of its useful time feeding matrix-multiply hardware;
- a memory copy can approach memory-system limits while doing very little arithmetic;
- an elementwise kernel can be limited by memory traffic or instruction issue rather than Tensor Cores;
- a reduction can be constrained by dependencies and synchronization.

Therefore, “the GPU has unused compute” is not a complete diagnosis. Which compute? Which instruction path? Is memory already the bottleneck? Are there eligible warps? Is the kernel too small to expose enough device-wide parallelism?

Nsight Compute’s throughput model reflects this reality by reporting high-level compute and memory throughput relative to peak and exposing lower-level contributors.[3]

> A GPU is a collection of coupled subsystems. Performance tuning means finding the subsystem that limits useful progress for a specific workload, not trying to make every block in a diagram read 100%.

20 张量核心（Tensor Cores）：矩阵（Matrix） 矩阵乘加（MMA），不是魔法AI 核心（Cores）

对应原文第 24 页；点击查看相邻原文对照页

20 张量核心（Tensor Cores）：矩阵（Matrix） 矩阵乘加（MMA），不是魔法 AI 核心（Cores） 张量核心（Tensor Cores） 是用于 矩阵（Matrix） 乘法累加 (矩阵乘加（MMA）) 运算的专用硬件。 NVIDIA 的编程接口公开了以下形式的概念操作

支持的图块形状、操作数格式、累加（Accumulation） 格式和执行范围取决于 GPU 代际（GPU Generation）。[7,11]

张量核心（Tensor Cores） 对于 AI 来说非常重要，因为 训练（Training） 和 推理（Inference） 包含大型 GEMM。但它们不会加速 模型（Model） 中的每条指令。索引、控制流、多次归约、元素转换、内存移动、同步（Synchronization） 和主机端工作使用其他资源。

“张量核心（Tensor Core）”一词也隐藏了代际的（Generational）的变化。早期的 CUDA 接口公开了 线程束（Warp） 级 矩阵（Matrix） 操作。 Hopper 为某些高 性能（Performance） 路径引入了异步 warpgroup 矩阵乘加（MMA） 指令，其中四个连续的 线程束（Warps） 作为一个 warpgroup 进行协作。数据中心 Blackwell SM100 为 矩阵（Matrix） 流水线（Pipeline） 的部分添加了较新的 tcgen05.矩阵乘加（MMA） 指令和 张量（Tensor） 内存 (TMEM)。[6,11]

这些差异是围绕持久原则的实现细节：当工作负载以受支持的 矩阵（Matrix） 切片和数据格式表示并且周围的 数据移动（Data Movement） 保持单元供电时，专用 矩阵（Matrix） 硬件可实现高 吞吐量（Throughput）。

张量核心（Tensor Cores） 提高了支持的 矩阵（Matrix） 算术的上限。它们不会从 AI 工作负载中删除 内存流量（Memory Traffic）、启动开销（Launch Overhead）、同步（Synchronization） 或非 矩阵（Matrix） 工作。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

20 Tensor Cores: matrix MMA, not magic AI cores

Tensor Cores are specialized hardware for matrix multiply-accumulate (MMA) operations. NVIDIA's programming interfaces expose operations conceptually of the form

$$D = A \times B + C.$$

The supported tile shapes, operand formats, accumulation formats, and execution scope depend on GPU generation.[7,11]

Tensor Cores matter enormously for AI because training and inference contain large GEMMs. But they do not accelerate every instruction in a model. Indexing, control flow, many reductions, elementwise transformations, memory movement, synchronization, and host-side work use other resources.

The word "Tensor Core" also hides generational change. Earlier CUDA interfaces exposed warp-level matrix operations. Hopper introduced asynchronous warpgroup MMA instructions for certain high-performance paths, where four contiguous warps cooperate as a warpgroup. Datacenter Blackwell SM100 adds newer tcgen05.mma instructions and Tensor Memory (TMEM) for parts of the matrix pipeline.[6,11]

These differences are implementation details around a durable principle: specialized matrix hardware achieves high throughput when the workload is expressed in supported matrix tiles and data formats and when the surrounding data movement keeps the units fed.

> Tensor Cores raise the ceiling for supported matrix arithmetic. They do not remove memory traffic, launch overhead, synchronization, or non-matrix work from an AI workload.

21 精度（Precision） 更改可用的执行路径

对应原文第 25 页；点击查看相邻原文对照页

21 精度（Precision） 更改可用执行路径 峰值算术 吞吐量（Throughput） 取决于数据类型。 GPU 对于 FP64、32 位浮点（FP32）、TF32、BF16、FP16、FP8、整数格式和较新的窄格式具有非常不同的理论速率。 矩阵（Matrix） 硬件还支持输入和 累加（Accumulation） 精度（Precision） 的特定组合，具体组合因 代际（Generation） 而异。[5,6,7,10]

这意味着诸如“X TFLOPs”之类的规范是不完整的，除非它至少标识：

- 操作类；

- 输入数据类型；

- 累加（Accumulation） 模式；

- 稠密与结构化稀疏假设（如果适用）；

- 硬件代际（Generation）。

精度（Precision） 不仅仅是一个 性能（Performance） 旋钮。它改变数字行为，内存占用（Memory Footprint），内存流量（Memory Traffic），有时还改变哪些执行指令是合法的。 FP16/BF16 训练（Training） 内核（Kernel） 可以使用 FP64 科学 内核（Kernel） 不可用的 矩阵（Matrix） 路径。 FP8 路径可能需要 缩放（Scaling） 和错误管理技术，而更高级别的 精度（Precision） 路径则不需要。

对于系统推理，请将两个声明分开：硬件支持高 吞吐量（Throughput） 低精度路径，并且此 模型（Model） 或 内核（Kernel） 可以安全有效地使用它。首先是建筑学。第二个取决于软件和数字要求。

21 Precision changes the available execution path

Peak arithmetic throughput depends on data type. A GPU can have very different theoretical rates for FP64, FP32, TF32, BF16, FP16, FP8, integer formats, and newer narrow formats. Matrix hardware also supports specific combinations of input and accumulation precision that vary by generation.[5,6,7,10]

This means a specification such as “X TFLOPs” is incomplete unless it identifies at least:

- the operation class;
- input data type;
- accumulation mode;
- dense vs structured-sparse assumptions where applicable;
- hardware generation.

Precision is not only a performance knob. It changes numerical behavior, memory footprint, memory traffic, and sometimes which execution instructions are legal. An FP16/BF16 training kernel may use matrix paths unavailable to an FP64 scientific kernel. An FP8 path may require scaling and error-management techniques that a higher-precision path does not.

For systems reasoning, keep two claims separate: the hardware supports a high-throughput lowprecision path and this model or kernel can safely and efficiently use it. The first is architectural. The second depends on software and numerical requirements.

22 通用矩阵乘法（GEMM） 如何映射到 GPU 层次结构

对应原文第 26 页；点击查看相邻原文对照页

22 通用矩阵乘法（GEMM） 如何映射到 GPU 层次结构 矩阵乘法（Matrix Multiplication） 是一个有用的架构示例，因为高 性能（Performance） 实现有意镜像 GPU 层次结构。 CUTLASS 将 通用矩阵乘法（GEMM） 描述为块的层次结构：设备范围的工作被分解为线程块块，然后是 线程束（Warp）/warpgroup 块，然后是线程或矩阵指令片段。 [11]

简化的数据流是：

全局 / HBM 块 共享内存块 寄存器（Register） / 矩阵（Matrix） 片段 矩阵乘加（MMA） / 数学管道 分块（Tiling） 的要点是重用。从 全局内存（Global Memory） 加载的图块在被逐出之前可以在许多乘法累加操作中重复使用。 寄存器（Registers） 将累加器或片段保存在靠近数学单元的位置。 共享内存（Shared Memory） 提供合作暂存区。[11]

现代 GPU 代际添加了更多异步机制以将 数据移动（Data Movement） 与 矩阵（Matrix） 计算重叠。 Hopper 的 WGMMA/TMA 生态系统和数据中心 Blackwell SM100 的 tcgen05/TMEM 路径就是示例。 [5,6,11]

数学 通用矩阵乘法（GEMM） C = AB 没有描述任何这些。实施时间表决定了瓷砖的居住位置、何时移动、合作多少个 线程束（Warps） 以及是否持续供应昂贵的 矩阵（Matrix） 单元。方程式和执行之间的差距是人工智能系统工程的中心主题之一。

22 How GEMM maps onto the GPU hierarchy

Matrix multiplication is a useful architectural example because high-performance implementations deliberately mirror the GPU hierarchy. CUTLASS describes GEMM as a hierarchy of tiles: devicewide work is decomposed into thread-block tiles, then warp/warpgroup tiles, then thread or matrixinstruction fragments.[11]

A simplified dataflow is:



The point of tiling is reuse. A tile loaded from global memory can be reused across many multiplyaccumulate operations before being evicted. Registers hold accumulators or fragments close to the math units. Shared memory provides a cooperative staging area.[11]

Modern generations add more asynchronous machinery to overlap data movement with matrix computation. Hopper's WGMMA/TMA ecosystem and datacenter Blackwell SM100's tcgen05/TMEM paths are examples.[5,6,11]

The mathematical GEMM $C = AB$ does not describe any of this. The implementation schedule determines where tiles live, when they move, how many warps cooperate, and whether the expensive matrix units are continuously supplied. That gap between equation and execution is one of the central themes of AI systems engineering.

23 指令级和线程级并行

对应原文第 27 页；点击查看相邻原文对照页

23 指令级和线程级并行性 GPU 可以使用多种规模的并行性来隐藏 延迟（Latency）。

线程级并行性 (TLP) 提供了许多 线程束（Warps）。如果 线程束（Warp） A 停止，调度程序可能会从 线程束（Warp） 发出

指令级并行性 (ILP) 在一个线程/线程束（Warp） 内提供独立操作。如果一个依赖链正在等待，则另一条独立指令可能可用。

这些机制相互作用。高 占用率（Occupancy） 会增加可贡献 TLP 的 线程束（Warps） 池。丰富的 ILP 可以减少保持管道繁忙所需的 驻留（Resident） 线程束（Warps） 数量。这是低 占用率（Occupancy） 不会自动变慢的原因之一。 NVIDIA 的最佳实践指南明确指出，即使在较低的 占用率（Occupancy） 下，足够的 ILP 有时也可以覆盖 延迟（Latency）。[2]

依赖性很重要，因为在结果准备好之前指令无法使用该结果。记忆依赖只是来源之一；算术管道也有 延迟（Latency）。安排尝试通过独立工作来填补空白。[2,3]

实际结果是 性能（Performance） 无法仅根据线程数来预测。如果一万个逻辑线程都达到相同的依赖模式，它们仍然可能会暴露出较差的就绪工作行为。

相反，精心流水线化的 内核（Kernel） 可以使专用资源忙于较小的 驻留（Resident） 集。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

23 Instruction-level and thread-level parallelism

A GPU can hide latency using parallelism at more than one scale.

Thread-level parallelism (TLP) provides many warps. If Warp A stalls, the scheduler may issue from Warp B.

Instruction-level parallelism (ILP) provides independent operations within one thread/warp. If one dependency chain is waiting, another independent instruction may be available.

These mechanisms interact. High occupancy increases the pool of warps that can contribute TLP. Rich ILP can reduce the number of resident warps required to keep pipelines busy. This is one reason low occupancy is not automatically slow. NVIDIA's Best Practices Guide explicitly notes that sufficient ILP can sometimes cover latency even at lower occupancy.[2]

Dependencies matter because an instruction cannot consume a result before that result is ready. Memory dependence is only one source; arithmetic pipelines also have latency. Scheduling attempts to fill the gaps with independent work.[2,3]

The practical consequence is that performance cannot be predicted from thread count alone. Ten thousand logical threads may still expose poor ready-work behavior if they all hit the same dependency pattern. Conversely, a carefully pipelined kernel can keep specialized resources busy with a smaller resident set.

24 同步（Synchronization） 可以暴露空闲时间

对应原文第 28 页；点击查看相邻原文对照页

24 同步（Synchronization） 可能会暴露空闲时间屏障对于“加载共享内存块，然后让每个线程使用它”等模式的正确性至关重要。但障碍可能会暴露不平衡。如果一些 线程束（Warps） 提前到达障碍，而其他人仍在工作，则早期的 线程束（Warps） 会等待。

因此，同步（Synchronization） 成本不仅仅是指令本身。它包括当线程等待最慢的所需参与者时变得不可用的有用工作量。

现代 CUDA 还提供 线程束（Warp） 级 同步（Synchronization）、协作组、异步屏障和特定于体系结构的机制，允许更复杂的生产者/消费者管道。 [1]

High-性能（Performance） AI 内核（Kernels） 使用这些工具来重叠阶段，但基本规则仍然存在：同步（Synchronization） 在工作人员之间创建依赖关系。计划必须尊重这些依赖性，并且不必要的 同步（Synchronization） 可能会减少可用的并行性。

如果 线程束（Warps） 中的大多数都在同一个 同步（Synchronization） 点等待，那么具有许多 驻留（Resident） 线程束（Warps） 的 内核（Kernel） 的 延迟（Latency） 隐藏效果仍然很差。 驻留状态（Residency） 仅在创建就绪工作时才有用。

线程协作，因此某些 内核（Kernels） 需要 同步（Synchronization）。 CUDA 的 __syncthreads() 是块范围的屏障：块中每个未退出的线程必须到达相同的屏障实例或退出，然后等待线程才能继续。它还提供参与线程之间所需的内存排序。[1]

24 Synchronization can expose idle time

Threads cooperate, so some kernels require synchronization. CUDA's `__syncthreads()` is a block-wide barrier: every non-exited thread in the block must reach the same barrier instance, or exit, before waiting threads can continue. It also provides the required memory ordering among participating threads.[1]

Barriers are essential for correctness in patterns such as “load a shared-memory tile, then let every thread consume it.” But a barrier can expose imbalance. If some warps reach the barrier early while others are still doing work, the early warps wait.

Synchronization cost is therefore not just the instruction itself. It includes the amount of useful work that becomes unavailable while threads wait for the slowest required participants.

Modern CUDA also provides warp-level synchronization, cooperative groups, asynchronous barriers, and architecture-specific mechanisms that allow more sophisticated producer/consumer pipelines.[1]

High-performance AI kernels use these tools to overlap stages, but the fundamental rule remains: synchronization creates dependencies between workers. The schedule must respect those dependencies, and unnecessary synchronization can reduce available parallelism.

> A kernel with many resident warps can still have poor latency hiding if most of those warps are waiting at the same synchronization point. Residency is useful only when it creates ready work.

25 内核（Kernel） 启动并异步执行

对应原文第 29 页；点击查看相邻原文对照页

25 内核（Kernel） 启动和异步执行 CUDA 内核（Kernel） 启动相对于主机线程是异步的：将 内核（Kernel） 排队后，CPU 可以继续，而设备工作仍处于挂起状态。需要结果或想要测量完成的 内核（Kernel） 时间的代码必须在适当的点同步。[1]

这会影响 性能（Performance） 和测量。除非正确处理 同步（Synchronization），否则在启动时放置的简单 CPU 计时器可以测量大部分 启动开销（Launch Overhead） 而不是 内核（Kernel） 运行时间。因此，CUDA 的最佳实践指南将 CPU 时序与 GPU 事件时序区分开来，并强调 API 的异步行为。 [2]

当工作被分割成许多微小的 内核（Kernels） 时，启动开销（Launch Overhead） 变得尤其重要。 GPU 可能具有巨大的算术能力，但如果每次启动执行的工作太少，则主机 分派（Dispatch） 和 内核（Kernel） 边界可能会在 实际耗时（Wall-clock Time） 中变得可见。 CUDA 图和融合是解决此问题的部分后续系列主题。

当资源和依赖性允许时，独立流可以重叠。较新的 GPU 机制还可以将 数据移动（Data Movement） 与 内核（Kernels） 内部的计算重叠。[1,5,11]

主机和设备执行解耦；仅在正确性或测量需要时进行同步。

25 Kernel launches and asynchronous execution

CUDA kernel launches are asynchronous with respect to the host thread: after enqueueing a kernel, the CPU can continue while device work is still pending. Code that needs the result, or wants to measure completed kernel time, must synchronize at the appropriate point.[1]

This affects both performance and measurement. A naive CPU timer placed around a launch can measure mostly launch overhead rather than kernel runtime unless synchronization is handled correctly. CUDA's Best Practices Guide therefore distinguishes CPU timing from GPU event timing and emphasizes the asynchronous behavior of the API.[2]

Launch overhead becomes especially relevant when work is fragmented into many tiny kernels. The GPU may have enormous arithmetic capacity, but if each launch performs too little work, host dispatch and kernel boundaries can become visible in wall-clock time. CUDA Graphs and fusion are later-series topics that address parts of this problem.

Independent streams can overlap when resources and dependencies permit. Newer GPU mechanisms can also overlap data movement with computation inside kernels.[1,5,11]

Host and device execution are decoupled; synchronize only where correctness or measurement requires it.

26“GPU 利用率（GPU Utilization）”的实际含义是什么

对应原文第 30 页；点击查看相邻原文对照页

26“GPU 利用率（GPU Utilization）”的实际含义 在 NVML 中，常见的设备 GPU 利用率（GPU Utilization） 值（Value） 是一种活动时间统计数据：在每个样本窗口中，它记录 GPU 至少执行一个 内核（Kernel） 的时间份额。[4]

该指标并不意味着：

- 100% 的算术单元繁忙；

- 张量核心（Tensor Cores） 已饱和；

- HBM 带宽已饱和；

- 每个流式多处理器（SM） 都已满员；

- 工作负载接近最佳吞吐量（Throughput）。

内核（Kernel） 可以连续执行，但仍然会留下大部分 GPU 未得到充分利用。例如，它可能只使用 SM 的一小部分，发出效率低下，或者受到一个 流水线（Pipeline） 的限制，而其他资源则闲置。

NVML 的内存字段同样是基于时间的：它记录发生全局设备内存读取或写入的样本窗口的份额。它不应被解释为实现的带宽除以峰值带宽。[4]

为了优化，内核（Kernel） 分析器（例如 Nsight Compute）公开了更具体的资源 - 吞吐量（Throughput）、占用率（Occupancy）、调度程序、缓存和内存指标。 [3]

仪表板可以正确显示 100%“GPU 利用率（GPU Utilization）”，而 内核（Kernel） 距离对其工作负载至关重要的 性能（Performance） 上限还很远。

26 What “GPU utilization” actually means

In NVML, the commonly surfaced device gpu utilization value is an activity-time statistic: over each sample window, it records the share of time for which the GPU had at least one kernel executing.[4]

That metric does not mean:

- 100% of arithmetic units were busy;
- Tensor Cores were saturated;
- HBM bandwidth was saturated;
- every SM was fully occupied;
- the workload was close to optimal throughput.

A kernel can execute continuously and still leave large parts of the GPU underused. For example, it might use only a small subset of SMs, issue inefficiently, or be limited by one pipeline while other resources sit idle.

NVML’s memory field is likewise time-based: it records the share of the sample window in which global device-memory reads or writes were occurring. It should not be interpreted as achieved bandwidth divided by peak bandwidth.[4]

For optimization, kernel profilers such as Nsight Compute expose more specific resource-throughput, occupancy, scheduler, cache, and memory metrics.[3]

A dashboard can correctly show 100% “GPU utilization” while a kernel is still far from the performance ceiling that matters for its workload.

27 占用率（Occupancy），活动性和吞吐量（Throughput）不同

对应原文第 31 页；点击查看相邻原文对照页

27 占用率（Occupancy）、activity 和 吞吐量（Throughput） 不同 GPU 工具中可以出现几个百分比，它们回答不同的问题。

指标系列 它试图描述什么 占用率（Occupancy） 驻留（Resident） 线程束（Warp） 相对于架构限制的分数 NVML GPU 利用率（GPU Utilization） 与 内核（Kernel） 执行的示例窗口的份额 活动 流式多处理器（SM） / 流水线（Pipeline） 活动 单元是否在测量周期内处于活动状态 合格/已发布线程束（Warps） 调度程序准备情况和实际问题机会，与 驻留状态（Residency） 分开 吞吐量（Throughput） 与相对于特定架构上限的峰值活动 Nsight Compute 的光速分析报告计算和内存 吞吐量（Throughput） 作为适当峰值速率的百分比。其调度程序部分分别报告活动的、合格的和已发布的 线程束（Warp） 行为。 [3]

这些数字可以不一致，但并不矛盾。 内核（Kernel） 可能具有 100% NVML 利用率，因为它连续运行；50% 占用率（Occupancy） 利用率，因为 寄存器（Register） 使用限制 驻留状态（Residency）；高内存吞吐量（Throughput），因为 HBM 接近其上限；适度算术 吞吐量（Throughput），因为 内核（Kernel）是带宽受限（Bandwidth-bound）。

该组合可以描述高效的 内核（Kernel）。如果内存已经成为限制资源，那么尝试提高算术利用率将无济于事。

只有在您询问指标测量的数量、时间间隔以及相对于哪个分母之后，指标才有意义。

27 Occupancy, activity, and throughput are different

Several percentages can appear in GPU tooling, and they answer different questions.

Metric family	What it is trying to describe
occupancy	resident warp fraction relative to the architectural limit
NVML GPU utilization	share of the sample window with kernel execution
active SM / pipeline activity	whether a unit was active over measured cycles
eligible / issued warps	scheduler readiness and actual issue opportunities, separate from residency
throughput vs peak	achieved activity relative to an architecture-specific ceiling

Nsight Compute's Speed Of Light analysis reports compute and memory throughput as percentages of appropriate peak rates. Its scheduler sections separately report active, eligible, and issued warp behavior.[3]

These numbers can disagree without contradiction. A kernel might have 100% NVML utilization because it runs continuously, 50% occupancy because register use limits residency, high memory throughput because HBM is near its ceiling, and modest arithmetic throughput because the kernel is bandwidth-bound.

That combination could describe an efficient kernel. Trying to raise arithmetic utilization would not help if memory is already the limiting resource.

> Metrics are only meaningful after you ask what quantity they measure, over what time interval, and relative to which denominator.

28 峰值 TFLOP 与实现的 吞吐量（Throughput）

对应原文第 32 页；点击查看相邻原文对照页

28 峰值 TFLOP 与实现的 吞吐量（Throughput） 峰值 TFLOP 是特定操作和指定假设下的 精度（Precision） 的硬件上限。这不是应用程序 吞吐量（Throughput）。[2,5,6]

为了接近峰值 矩阵（Matrix） 吞吐量（Throughput），工作负载必须提供支持的数据类型和切片形状、足够的并行工作、高操作数重用、足够的指令发出以及保持 矩阵（Matrix） 单元供应的数据 流水线（Pipeline）。真正的 模型（Models） 还执行非 矩阵（Matrix） 内核（Kernels） 并产生内存移动、同步（Synchronization）、通信、启动和框架开销。

基准测试（Benchmark） 范围很重要。测量一个 通用矩阵乘法（GEMM） 内核（Kernel） 的微基准不能直接与 推理服务（Serving） 系统的 词元（Tokens）/s 进行比较。一种数据类型所达到的 TFLOP/s 数无法与另一种数据类型的峰值相比较。供应商的稀疏峰值与密集峰值不同。

Nsight Compute 的 吞吐量（Throughput） 指标非常有用，因为它们根据特定于架构的峰值速率显式标准化测量的子系统活动，同时仍然要求工程师确定哪些资源实际上限制了 内核（Kernel）。 [3]

切勿读取峰值数字，因为“GPU 将为我的 模型（Model） 提供此值。”Peak是特定硬件路径的上限；应用程序 吞吐量（Throughput） 包含生成结果所需的所有内容。

对于相同的操作 精度（Precision） 和计数约定，硬件峰值是持续算术 吞吐量（Throughput） 的上限。端到端应用程序 性能（Performance） 还反映内存移动、同步（Synchronization）、通信、内核（Kernel） 启动和非 矩阵（Matrix） 工作。

28 Peak TFLOPs vs achieved throughput

Peak TFLOPs is a hardware ceiling for a particular operation and precision under specified assumptions. It is not application throughput.[2,5,6]

To approach peak matrix throughput, a workload must provide supported data types and tile shapes, enough parallel work, high operand reuse, sufficient instruction issue, and a data pipeline that keeps the matrix units supplied. Real models also execute non-matrix kernels and incur memory movement, synchronization, communication, launch, and framework overhead.

For the same operation, precision, and counting convention, hardware peak is a ceiling on sustained arithmetic throughput. End-to-end application performance additionally reflects memory movement, synchronization, communication, kernel launches, and non-matrix work.

Benchmark scope matters. A microbenchmark that measures one GEMM kernel is not directly comparable to tokens/s for a serving system. An achieved TFLOP/s number for one dtype is not comparable to another dtype's peak. A vendor's sparse peak is not the same as dense peak.

Nsight Compute's throughput metrics are useful because they explicitly normalize measured subsystem activity against architecture-specific peak rates, while still requiring the engineer to identify which resource is actually limiting the kernel.[3]

> Never read a peak number as "the GPU will deliver this for my model." Peak is a ceiling for a specific hardware path; application throughput includes everything required to produce the result.

29 计算受限（Compute-bound） 和 内存受限（Memory-bound） 是工作负载属性

对应原文第 33 页；点击查看相邻原文对照页

29 计算受限（Compute-bound） 和 内存受限（Memory-bound） 是工作负载属性 内核（Kernel） 需要算术属性和 数据移动（Data Movement） 属性。一个有用的一阶数量是 算术强度（Arithmetic Intensity）

字节在该内存级别上移动。

原始 Roofline 公式使用片外 内存流量（Memory Traffic） (运算强度（Operational Intensity）) 将可达到的性能（Performance） 与峰值计算和 内存带宽（Memory Bandwidth） 相关联。层次结构变体可以将相同的推理风格应用于其他级别。[13]

高强度的 内核（Kernels） 数据重用足以使算术 吞吐量（Throughput） 成为极限。低强度 内核（Kernels） 移动许多字节以进行相对较少的计算，并且更有可能受到 内存带宽（Memory Bandwidth） 的约束。确切的边界取决于硬件。[13]

AS01 中有两个注意事项很重要。

1. “内存受限（Memory-bound）”并不意味着“使用大量内存容量”。这意味着进度受到测量工作负载下内存系统资源的限制。

2. 内核（Kernel） 的名称并非永久为 计算受限（Compute-bound） 或 内存受限（Memory-bound）。Shape、dtype、批大小（Batch Size）、缓存行为、融合和 GPU 代际（GPU Generation） 可以改变瓶颈。

完整的 屋顶线模型（Roofline Model） 属于 AS04。对于架构来说，这个教训已经足够了：GPU 的计算和内存系统提供了单独的上限。提高非限制上限并不一定会使工作负载更快。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

29 Compute-bound and memory-bound are workload properties

A kernel needs both arithmetic and data movement. One useful first-order quantity is arithmetic intensity for a chosen memory level:

$$I = \frac{\text{operations}}{\text{bytes moved across that memory level}}.$$

The original Roofline formulation uses off-chip memory traffic (operational intensity) to relate attainable performance to peak compute and memory bandwidth. Hierarchical variants can apply the same style of reasoning to additional levels.[13]

High-intensity kernels reuse data enough that arithmetic throughput can become the limiting ceiling. Low-intensity kernels move many bytes for relatively little computation and are more likely to be constrained by memory bandwidth. The exact boundary depends on the hardware.[13]

Two caveats matter in AS01.

1. “Memory-bound” does not mean “uses lots of memory capacity.” It means progress is limited by a memory-system resource under the measured workload.

2. A kernel is not permanently compute-bound or memory-bound by name. Shape, dtype, batch size, cache behavior, fusion, and GPU generation can change the bottleneck.

The full Roofline model belongs in AS04. For architecture, the lesson is enough: a GPU’s compute and memory systems provide separate ceilings. Improving the non-limiting ceiling does not necessarily make a workload faster.

30 个 AI 工作负载对 GPU 的不同部分产生压力

对应原文第 34 页；点击查看相邻原文对照页

30 个 AI 工作负载对 GPU 的不同部分施加压力“AI 工作负载”不是一种 性能（Performance） 机制。

大型密集GEMM可以通过分块（Tiling）实现高算术强度（Arithmetic Intensity）并重用，使矩阵（Matrix） 吞吐量（Throughput）和周围数据流水线（Pipeline）成为中心。

Elementwise 内核（Kernels） 可能只对每个元素执行少量操作，因此花费大量成本来移动数据。融合可以通过避免中间张量的往返来提供帮助。

归约和 归一化（Normalization） 引入了与令人尴尬的并行元素工作不同的依赖关系和集体行为。

嵌入（Embedding） 或索引工作负载可能具有不规则的访问模式，从而减少局部性或 合并访问（Coalescing）。

注意力机制（Attention） 包含 矩阵（Matrix） 乘法、Softmax 归一化（Softmax）/归约行为以及潜在的大中间流量。 FlashAttention 证明，在不改变 稠密注意力（Dense Attention） 数学结果的情况下更改 IO 调度可以显着改善 性能（Performance）。[12]

自回归 推理服务（Serving） 将形状从长提示 预填充（Prefill） 更改为一个/几个 查询（Query） 解码步骤，因此相同的 模型（Model） 可以在不同阶段对硬件施加不同的压力。确切的瓶颈取决于 批处理（Batching）、上下文长度、内核（Kernel） 选择和系统设计。

这种多样性就是为什么 模型（Model） 级“GPU 利用率（GPU Utilization）”数字对于认真的诊断来说过于粗糙。 推理服务（Serving） 跟踪可能会在具有完全不同瓶颈的 内核（Kernels） 之间交替。

30 AI workloads stress different parts of the GPU

“AI workload” is not one performance regime.

Large dense GEMMs can achieve high arithmetic intensity through tiling and reuse, making matrix throughput and the surrounding data pipeline central.

Elementwise kernels may perform only a few operations per element and therefore spend much of their cost moving data. Fusion can help by avoiding round-trips of intermediate tensors.

Reductions and normalization introduce dependencies and collective behavior that differ from embarrassingly parallel elementwise work.

Embedding or indexing workloads can have irregular access patterns that reduce locality or coalescing.

Attention contains matrix multiplications, softmax/reduction behavior, and potentially large intermediate traffic. FlashAttention demonstrates that changing the IO schedule without changing dense attention’s mathematical result can materially improve performance.[12]

Autoregressive serving changes shape from long-prompt prefill to one/few-query decode steps, so the same model can stress hardware differently across phases. The exact bottleneck depends on batching, context length, kernel choice, and system design.

This variety is why a model-level “GPU utilization” number is too coarse for serious diagnosis. A serving trace may alternate between kernels with entirely different bottlenecks.

31 NVIDIA 与 AMD：名称不同，原则不变

对应原文第 35 页；点击查看相邻原文对照页

31 NVIDIA 与 AMD：名称不同，原则不变 CUDA 术语并不是 GPU 的定义。 AMD 公开了类似的广泛概念，但名称不同，宽度和机制也不同。 HIP 明确警告可移植代码不要采用固定的 线程束（Warp） 大小。 AMD 将硬件分组称为 波前（Wavefront）；其大小因架构而异，而 NVIDIA CUDA 的 线程束（Warp） 大小为 32。[8]

概念 NVIDIA 语言 AMD 语言执行引擎 流式多处理器（Streaming Multiprocessor） (流式多处理器（SM）) 计算单元 (CU) 线程组 线程束（Warp） 波前（Wavefront） / HIP 抽象程序员管理的片上存储中的 线程束（Warp） 共享内存（Shared Memory） LDS (本地数据共享) 矩阵（Matrix） 加速 张量核心（Tensor Core） / 矩阵乘加（MMA） 系列 矩阵（Matrix） 核心（Core） / MFMA 系列编程堆栈 CUDA ROCm / HIP AMD 当前的 性能（Performance） 指南描述了相同的高级 占用率（Occupancy） 权衡：寄存器（Registers） 和共享/LDS 资源限制并发块/波，以及更高占用率（Occupancy） 可以帮助隐藏 延迟（Latency）。[9]

因此，架构课程是可移植的：公开并行工作，在硬件组中调度它，提供每个线程和每个块状态，通过层次结构移动数据，并提供专门的执行单元。名称、确切宽度、资源限制和说明是特定于供应商和 代际（Generation） 的。

31 NVIDIA vs AMD: names differ, principles persist

CUDA terminology is not the definition of a GPU. AMD exposes similar broad concepts with different names and some different widths and mechanisms. HIP explicitly warns portable code not to assume one fixed warp size. AMD calls the hardware grouping a wavefront; its size varies by architecture, while NVIDIA CUDA's warp size is 32.[8]

Concept	NVIDIA language	AMD language
execution engine	Streaming Multiprocessor (SM)	Compute Unit (CU)
thread group	warp	wavefront / warp in HIP abstraction
programmer-managed on-chip storage	shared memory	LDS (Local Data Share)
matrix acceleration	Tensor Core / MMA families	Matrix Core / MFMA families
programming stack	CUDA	ROCm / HIP

AMD's current performance guidance describes the same high-level occupancy trade-off: registers and shared/LDS resources constrain concurrent blocks/waves, and higher occupancy can help hide latency.[9]

The architectural lesson is therefore portable: expose parallel work, schedule it in hardware groups, provision per-thread and per-block state, move data through a hierarchy, and feed specialized execution units. The names, exact widths, resource limits, and instructions are vendorand generation-specific.

32 代架构改变了机制

对应原文第 36 页；点击查看相邻原文对照页

32 代架构改变了机制 如果一本关于 GPU 架构的手册将 代际（Generation） 的实现与永恒的理论相混淆，它就会变得过时。持久原则是 线程束（Warps）/波前、驻留状态（Residency）、调度、数据移动（Data Movement）、执行管道和资源限制。机制不断发展。

Hopper 为高 性能（Performance） 内核（Kernels） 引入了 TMA、线程块集群和线程束组级异步 MMA。[5,11]

数据中心 Blackwell SM100 引入了 tcgen05.矩阵乘加（MMA） 张量核心（Tensor Core） 指令以及这些 矩阵（Matrix） 管道使用的不同 张量（Tensor） 内存 (TMEM)。 CUTLASS 将这些视为架构条件功能，而不是通用 Blackwell 或 CUDA 行为，并明确区分数据中心 SM100 与 GeForce Blackwell SM120。[6,11]

AMD CDNA 系列同样发展了 矩阵（Matrix） 格式、计算单元结构、封装和内存系统。例如，CDNA 3 扩展了 矩阵（Matrix） 核心（Core） 数据类型支持，并成为 MI300 加速器系列的基础。 [10]

这种演变就是为什么本手册中标记了特定于体系结构的详细信息，也是为什么后来的 性能（Performance） 工作必须与实际部署的 GPU 和软件版本相关联的原因。

不要将框图记住为“GPU”。记住不变量，然后查阅 代际（Generation） 特定实现的架构文档。

32 Architecture generations change the mechanisms

A handbook on GPU architecture can become obsolete if it confuses one generation's implementation with timeless theory. The durable principles are warps/wavefronts, residency, scheduling, data movement, execution pipelines, and resource constraints. The mechanisms evolve.

Hopper introduced TMA, thread-block clusters, and warpgroup-level asynchronous MMA for high-performance kernels.[5,11]

Datacenter Blackwell SM100 introduced tcgen05.mma Tensor Core instructions and a distinct Tensor Memory (TMEM) used by those matrix pipelines. CUTLASS treats these as architecture-conditional capabilities rather than universal Blackwell or CUDA behavior, and explicitly distinguishes datacenter SM100 from GeForce Blackwell SM120.[6,11]

AMD CDNA generations likewise evolve matrix formats, compute-unit structure, packaging, and memory systems. CDNA 3, for example, extends Matrix Core datatype support and underlies the MI300 accelerator family.[10]

This evolution is why architecture-specific details in this handbook are labeled and why later performance work must be tied to the actual deployed GPU and software version.

> Do not memorize one block diagram as "the GPU." Memorize the invariants, then consult the architecture documentation for the generation-specific realization.

33 有效执行示例：61,440 个逻辑线程

对应原文第 37 页；点击查看相邻原文对照页

33 有效执行示例：61,440 个逻辑线程 考虑使用 240 个块启动的 CUDA 内核（Kernel） 每块 256 个线程 网格包含

逻辑线程。因为 NVIDIA 线程束（Warp） 包含 32 个线程，所以每个块包含

线程束（Warps），完整的网格包含

线程束（Warps） 的逻辑工作。这 1,920 个 线程束（Warps） 并不是同时全部是 驻留（Resident）。当资源可用时，块会在 SM 之间调度。一个 流式多处理器（SM） 上可以容纳多少个块取决于设备的架构限制以及该 内核（Kernel） 的资源要求，尤其是 寄存器（Registers） 和 共享内存（Shared Memory）。[1,2]

一旦块是 驻留（Resident），线程束（Warp） 调度程序就会选择 可调度线程束（Eligible Warps） 并向适当的执行路径发出指令。某些 线程束（Warps） 可能会等待内存或依赖项，而其他 线程束（Warps） 可能会继续。当块完成时，它们的资源被释放，以后的块可以变成 驻留（Resident）。

此示例区分了三个尺度：

- 61,440 个线程：逻辑并行工作；

- 1,920 线程束（Warps）：整个网格中的 NVIDIA 执行分组；

- 驻留（Resident） 线程束（Warps） 一次：受实际设备和内核（Kernel） 资源使用的限制。

算术很简单，但区别是基本的。启动大小描述了要完成的工作，而不是瞬时物理并行性。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

33 Worked execution example: 61,440 logical threads

Consider a CUDA kernel launched with

240 blocks	256 threads per block

The grid contains

$$240 \times 256 = 61{,}440$$

logical threads. Because an NVIDIA warp contains 32 threads, each block contains

$$256/32 = 8$$

warps, and the complete grid contains

$$240 \times 8 = 1{,}920$$

warps of logical work. Those 1,920 warps are not all resident at once. Blocks are dispatched across SMs as resources become available. How many blocks fit on one SM depends on the device's architectural limits and this kernel's resource requirements, especially registers and shared memory.[1,2]

Once blocks are resident, warp schedulers choose eligible warps and issue their instructions to the appropriate execution paths. Some warps may wait on memory or dependencies while others proceed. As blocks complete, their resources are freed and later blocks can become resident.

This example separates three scales:

- 61,440 threads: logical parallel work;
- 1,920 warps: NVIDIA execution groupings across the whole grid;
- resident warps at one time: bounded by the actual device and kernel resource use.

The arithmetic is simple, but the distinction is fundamental. Launch size describes work to be done, not instantaneous physical parallelism.

34 已工作 占用率（Occupancy） 示例：资源限制相互作用

对应原文第 38 页；点击查看相邻原文对照页

34 已工作的 占用率（Occupancy） 示例：资源限制相互作用 使用故意假设的 流式多处理器（SM），以便算术说明该机制，而无需假装描述特定 GPU。假设它最多允许 64 个 驻留（Resident） 线程束（Warps），有足够的块槽用于以下配置，并且没有额外的分配粒度限制。它包含 65,536 个 32 位 寄存器（Registers） 和 128 KiB 共享内存（Shared Memory）。

内核（Kernel） 每个块使用 256 个线程，每个线程 64 个 寄存器（Registers），每个块使用 32 KiB 共享内存（Shared Memory）。

256/32 = 8 线程束（Warps） 每块 寄存器（Register） 每块需求为

因此 寄存器（Registers） 最多允许 65,536/16,384 = 4 个块。

共享内存（Shared Memory） 也允许

因此，在这个简化的模型（Model）中，四个块是驻留（Resident），或者4×8 = 32 线程束（Warps），而占用率（Occupancy）是32/64 = 50%。

如果 共享内存（Shared Memory） 减少到 16 KiB，占用率（Occupancy） 将不会自动上升：寄存器（Registers） 仍会将 驻留状态（Residency） 限制在四个区块。优化非限制性资源不会改变任何事情。这正是 占用率（Occupancy） 分析必须识别活动约束而不是单独调整一个数字的原因。[2,3]

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

34 Worked occupancy example: resource limits interact

Use a deliberately hypothetical SM so the arithmetic illustrates the mechanism without pretending to describe a specific GPU. Suppose it allows at most 64 resident warps, has enough block slots for the configurations below, and no additional allocation-granularity limit. It contains 65,536 32-bit registers and 128 KiB of shared memory.

A kernel uses 256 threads per block, 64 registers per thread, and 32 KiB shared memory per block.

$$256/32 = 8 \text{ warps per block}$$

Register demand per block is

$$256 \times 64 = 16{,}384 \text{ registers},$$

so registers permit at most

$$65{,}536/16{,}384 = 4 \text{ blocks}.$$

Shared memory also permits

$$128 \text{ KiB}/32 \text{ KiB} = 4 \text{ blocks}.$$

Therefore four blocks are resident in this simplified model, or $4 \times 8 = 32$ warps, and occupancy is $32/64 = 50\%$. If shared memory were reduced to 16 KiB, occupancy would not automatically rise: registers would still cap residency at four blocks. Optimizing the non-limiting resource changes nothing. This is exactly why occupancy analysis must identify the active constraint rather than tune one number in isolation.[2,3]

35 种常见误解

对应原文第 39 页；点击查看相邻原文对照页

35 个常见误解“GPU 只是数千个微型 CPU”。不。线程、调度、内存和 吞吐量（Throughput） 设计根本不同。[1]

“更多的 CUDA 核心（CUDA Cores） 意味着相应更快的人工智能。”否。AI 内核（Kernels） 使用不同的管道、内存资源和精度； 核心（Core） 单独计数不是 性能（Performance） 模型（Model）。

“100% GPU 利用率（GPU Utilization） 表示 GPU 已饱和。”不会。NVML 通常显示的 值（Value） 是一种随时间变化的活动度量，而不是 饱和（Saturation） 分数。[4]

“张量核心（Tensor Cores） 加速整个 模型（Model）。”不会。它们会加速受支持的 矩阵（Matrix） 矩阵乘加（MMA） 操作。其他工作使用其他资源。[7,11]

“如果 模型（Model） 适合 HBM，内存问题就解决了。”不可以。容量、流量、带宽和 延迟（Latency） 是不同的约束。

“共享内存（Shared Memory） 只是一个缓存。”不会。它由程序显式管理，即使硬件与 L1 共享物理资源也是如此。[1]

“延迟（Latency） 隐藏会删除 延迟（Latency）。”不。它与等待间隔重叠其他工作。[2]

“AI 内核（Kernels） 是 计算受限（Compute-bound）。”有些是。其他是带宽、延迟（Latency）、问题、同步（Synchronization） 或启动限制，具体取决于形状和实现。

35 Common misconceptions

“A GPU is just thousands of tiny CPUs.”

No. The threading, scheduling, memory, and throughput design is fundamentally different.[1]

“More CUDA cores means proportionally faster AI.”

No. AI kernels use different pipelines, memory resources, and precisions; core count alone is not a performance model.

"100% GPU utilization means the GPU is saturated." No. NVML’s commonly displayed value is an activity-over-time measure, not a saturation score.[4]

“Tensor Cores accelerate the whole model.”

No. They accelerate supported matrix MMA operations. Other work uses other resources.[7,11]

“If the model fits in HBM, memory is solved.”

No. Capacity, traffic, bandwidth, and latency are different constraints.

“Shared memory is just a cache.”

No. It is explicitly managed by the program, even when hardware shares physical resources with L1.[1]

“Latency hiding removes latency.”

No. It overlaps other work with the waiting interval.[2]

“AI kernels are compute-bound.”

Some are. Others are bandwidth-, latency-, issue-, synchronization-, or launch-limited depending on shape and implementation.

36 实用的 理解框架（Mental Model）

对应原文第 40 页；点击查看相邻原文对照页

36 实用的 理解框架（Mental Model） 推理 GPU AI 工作负载的紧凑方法是：

1.暴露工作。 内核（Kernel） 定义了块和线程的网格。

2. 放置积木。硬件允许块进入 SM，但须遵守 寄存器（Register）、共享内存、线程、线程束（Warp） 和块限制。

3. 组建执行小组。 NVIDIA 块变为 32 线程 线程束（Warps）；其他供应商使用自己的分组规则。

4. 保持状态关闭。 寄存器（Registers） 和 共享内存（Shared Memory） 保持有用的工作状态，同时缓存减少层次结构中更下游的流量。

5. 安排准备情况。 线程束（Warp） 调度程序选择符合条件的工作，因此一个停止的依赖项不需要停止整个 流式多处理器（SM）。

6. 为正确的管道供电。标量/向量（Vector） 算术、加载/存储路径、矩阵（Matrix） 单元和其他资源处理不同的指令。

7. 有意地移动数据。 合并访问（Coalescing）、分块（Tiling）、缓存、重用和融合决定了有多少流量到达设备内存（Device Memory）。

8. 测量瓶颈。 占用率（Occupancy），利用率、带宽、问题效率和实现的计算是不同的指标。使用能回答当前问题的答案。

工程回报是一个更好的问题。不要问“为什么我的 GPU 没有达到 100%？”，而是问：

哪些资源或依赖项限制了此工作负载的有用进度，证据（Evidence） 显示了哪些内容？

该问题的范围从单个 CUDA 内核（Kernel） 到 注意力机制（Attention）、模型（Model） 训练（Training）、分布式执行和 LLM 推理服务（Serving）。它是理解人工智能系统其余部分的基础。

36 A practical mental model

A compact way to reason about a GPU AI workload is:

1. Expose work. A kernel defines a grid of blocks and threads.

2. Place blocks. Hardware admits blocks onto SMs subject to register, shared-memory, thread, warp, and block limits.

3. Form execution groups. NVIDIA blocks become 32-thread warps; other vendors use their own grouping rules.

4. Keep state close. Registers and shared memory hold useful working state while caches reduce traffic farther down the hierarchy.

5. Schedule readiness. Warp schedulers choose eligible work so one stalled dependency need not stall the entire SM.

6. Feed the right pipelines. Scalar/vector arithmetic, load/store paths, matrix units, and other resources handle different instructions.

7. Move data deliberately. Coalescing, tiling, caching, reuse, and fusion determine how much traffic reaches device memory.

8. Measure the bottleneck. Occupancy, utilization, bandwidth, issue efficiency, and achieved compute are different metrics. Use the one that answers the current question.

The engineering payoff is a better question. Instead of asking “why isn’t my GPU at 100%?”, ask:

Which resource or dependency limits useful progress for this workload, and what evidence shows that?

That question scales from a single CUDA kernel to attention, model training, distributed execution, and LLM serving. It is the foundation for the rest of Understanding AI Systems.

参考文献和进一步阅读

对应原文第 41 页；点击查看相邻原文对照页

参考文献和进一步阅读

[1] NVIDIA.CUDA 编程指南，最新文档，2026 年 9 月访问。https://docs.nvidia.com/cuda/cuda-programming-guide/

[2] 英伟达。 CUDA C++ 最佳实践指南，版本 13.4 文档，于 2026 年 9 月访问。https:

//docs.nvidia.com/cuda/cuda-c-best-practices-guide/

[3] NVIDIA.Nsight 计算：性能剖析（Profiling） 指南和用户指南，最新文档。
https://docs.nvidia.com/nsight-compute/

[4]英伟达。 NVML API 参考：nvmlUtilization_t。 https://docs.nvidia.com/deploy/nvml-api/api/structnvmlUtilization__t.html

[5] NVIDIA.Hopper 调整指南。 https://docs.nvidia.com/cuda/hopper-tuning-guide/

[6] NVIDIA.CUTLASS：Blackwell SM100 GEMM / tcgen05 功能。
https://docs.nvidia.com/cutlass/latest/media/docs/cpp/blackwell_functionity.html

[7]英伟达。 CUDA 编程指南 矩阵（Matrix） / WMMA API；另请参阅历史 张量核心（Tensor Core） 编程 模型（Model） 的图灵调整指南。 https://docs.nvidia.com/cuda/cuda-programming-guide/

[8] AMD.HIP C++ 语言扩展、warpSize 和跨通道行为、当前 ROCm 文档。 https:

//rocm.docs.amd.com/projects/HIP/en/latest/how-to/hip_cpp_language_extensions.html

[9] AMD.HIP 性能（Performance） 指南、占用率（Occupancy） 和资源指南、当前 ROCm 文档。 https:

//rocm.docs.amd.com/projects/HIP/en/latest/how-to/性能（Performance）_guidelines.html

[10] AMD.AMD CDNA 3 架构简介，白皮书。 https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/white-papers/amd-cdna-3-white-paper.pdf

[11] NVIDIA.CUTLASS 文档：高效 通用矩阵乘法（GEMM）、Warpgroup 矩阵乘加（MMA） 和 tcgen05 编程指南。

https://docs.nvidia.com/cutlass/latest/

[12] 道，T.；傅德义;埃尔蒙，S.；鲁德拉，A.； Ré，C.FlashAttention：具有 IO 感知功能的快速、内存高效的 精确注意力（Exact Attention）。 NeurIPS 2022。https://arxiv.org/abs/2205.14135

[13] 威廉姆斯，S.；沃特曼，A.； Patterson, D.Roofline：针对 浮点（Floating-point） 程序和多核架构的富有洞察力的可视化 性能（Performance） 模型（Model）。 UCB/EECS-2008-134； CACM 2009。
https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.html

References and further reading

[1] NVIDIA. *CUDA Programming Guide*, current documentation, accessed September 2026. https://docs.nvidia.com/cuda/cuda-programming-guide/

[2] NVIDIA. *CUDA C++ Best Practices Guide*, Release 13.4 documentation, accessed September 2026. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/

[3] NVIDIA. *Nsight Compute: Profiling Guide and User Guide*, current documentation. https://docs.nvidia.com/nsight-compute/

[4] NVIDIA. *NVML API Reference: nvmlUtilization_t*. https://docs.nvidia.com/deploy/nvml-api/api/structnvmlUtilization__t.html

[5] NVIDIA. *Hopper Tuning Guide*. https://docs.nvidia.com/cuda/hopper-tuning-guide/

[6] NVIDIA. *CUTLASS: Blackwell SM100 GEMMs / tcgen05 functionality*. https://docs.nvidia.com/cutlass/latest/media/docs/cpp/blackwell_functionality.html

[7] NVIDIA. CUDA Programming Guide matrix / WMMA APIs; see also the Turing Tuning Guide for the historical Tensor Core programming model. https://docs.nvidia.com/cuda/cuda-programming-guide/

[8] AMD. *HIP C++ Language Extensions*, warpSize and cross-lane behavior, current ROCm documentation. https://rocm.docs.amd.com/projects/HIP/en/latest/how-to/hip_cpp_language_extensions.html

[9] AMD. *HIP Performance Guidelines*, occupancy and resource guidance, current ROCm documentation. https://rocm.docs.amd.com/projects/HIP/en/latest/how-to/performance_guidelines.html

[10] AMD. *Introducing AMD CDNA 3 Architecture*, white paper. https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/white-papers/amd-cdna-3-white-paper.pdf

[11] NVIDIA. *CUTLASS Documentation*: Efficient GEMM, Warpgroup MMA, and tcgen05 programming guides. https://docs.nvidia.com/cutlass/latest/

[12] Dao, T.; Fu, D. Y.; Ermon, S.; Rudra, A.; Ré, C. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS 2022. https://arxiv.org/abs/2205.14135

[13] Williams, S.; Waterman, A.; Patterson, D. *Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures*. UCB/EECS-2008-134; CACM 2009. https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.html

参考图：针对哪个问题阅读什么内容

对应原文第 42 页；点击查看相邻原文对照页

参考图：针对哪个问题阅读什么内容问题最佳起始源当前CUDA线程/块/线程束（Warp）执行模型（Model）是什么？

CUDA 编程指南 [1] 占用率（Occupancy）、寄存器（Registers）、共享内存（Shared Memory） 和 延迟（Latency） 隐藏有何关系？

CUDA 最佳实践指南 [2] 性能剖析（Profiling） 中的有效、合格和已颁发的 线程束（Warps） 意味着什么？

Nsight Compute [3] 常见的 NVIDIA“GPU 利用率（GPU Utilization）”百分比到底衡量什么？

NVML nvmlUtilization_t [4] 哪些机制是 Hopper 特定的？ Hopper 调整指南 [5] 和 CUTLASS [11] 哪些机制将数据中心 Blackwell SM100 级路径与 SM120 区分开来？

CUTLASS Blackwell / tcgen05 文档 [6,11] 张量核心（Tensor Core） 矩阵（Matrix） 操作实际计算什么？

Turing/CUDA 矩阵（Matrix） 文档 [7] 为什么便携式 GPU 代码不应该假设跨供应商的一个 线程束（Warp） 宽度？

AMD HIP 语言扩展 [8] AMD 如何描述 占用率（Occupancy） 和 延迟（Latency） 隐藏？

AMD HIP 性能（Performance） 指南 [9] AMD CDNA 3 白皮书 [10] 高 性能（Performance） 通用矩阵乘法（GEMM） 如何映射到块、线程束（Warps）、共享内存（Shared Memory） 和 寄存器（Registers）？

CUTLASS 通用矩阵乘法（GEMM） 文档 [11] 为什么更少的 内存流量（Memory Traffic） 可以使相同的注意力机制（Attention） 数学运算更快？

FlashAttention [12] 计算上限和 内存带宽（Memory Bandwidth） 如何交互？

屋顶线模型（Roofline Model） [13] 实现支持和硬件细节的变化比架构原则更快。对于已部署的系统，请将确切的 GPU 代际（GPU Generation）、工具包、编译器、框架和 性能剖析器（Profiler） 版本的文档视为权威文档。

面向 AI 工程师、开发人员和认真学习者的简短技术手册 什么是 AMD AI 加速器架构示例？

Reference map: what to read for which question

Question	Best starting source
What is the current CUDA thread/block/warp execution model?	CUDA Programming Guide [1]
How are occupancy, registers, shared memory, and latency hiding related?	CUDA Best Practices Guide [2]
What do active, eligible, and issued warps mean in profiling?	Nsight Compute [3]
What exactly does the common NVIDIA “GPU utilization” percentage measure?	NVML nvmlUtilization_t [4]
Which mechanisms are Hopper-specific?	Hopper Tuning Guide [5] and CUTLASS [11]
Which mechanisms distinguish datacenter Blackwell SM100-class paths from SM120?	CUTLASS Blackwell / tcgen05 docs [6,11]
What does a Tensor Core matrix operation actually compute?	Turing/CUDA matrix documentation [7]
Why should portable GPU code not assume one warp width across vendors?	AMD HIP language extensions [8]
How does AMD describe occupancy and latency hiding?	AMD HIP performance guidance [9]
What is an AMD AI-accelerator architecture example?	AMD CDNA 3 white paper [10]
How does a high-performance GEMM map onto blocks, warps, shared memory, and registers?	CUTLASS GEMM documentation [11]
Why can less memory traffic make the same attention mathematics faster?	FlashAttention [12]
How do compute ceilings and memory bandwidth interact?	Roofline model [13]

Implementation support and hardware details change faster than the architectural principles. For deployed systems, treat the documentation for the exact GPU generation, toolkit, compiler, framework, and profiler version as authoritative.